

NRCM HACKATHON · PNT1 · LEVEL 1

AEGIS Twin

Software Requirements Specification

See every shipment. Simulate every shock. Survive every attack.

Standard	IEEE 830-lite
Version	1.0 — Level 1 baseline
Date	30 September 2026
Scope	FR-1...FR-25 · NFR-1...NFR-12 · architecture · DFD · ERD · prototype · roadmap

Contents

1. Introduction
2. Overall description
3. Specific requirements
4. Architecture
5. Data flow
6. Data model
7. User interface — clickable prototype
8. Engine verification
9. Technology stack
10. Roadmap and risks
11. Research grounding
12. Traceability matrix
13. Appendix A — Scenario DSL
14. Appendix B — Trust layer reason codes (initial set)

Logistics Network Digital Twin (PNT1) · NRCM Hackathon · Level 1

See every shipment. Simulate every shock. Survive every attack.

Field	Value
Document	Software Requirements Specification (IEEE 830-lite)
Version	1.1 — Level 1 baseline; §8 engine verification updated with Phase 3 measured results
Date	30 September 2026
Status	Baselined for Level 1; requirements are frozen, NFR values are targets to be validated in later phases
Prototype	Clickable high-fidelity prototype, <code>apps/web</code> (static mock data) — URL in the submission README
Companion artefacts	<code>docs/architecture/*</code> , <code>docs/dfd/*</code> , <code>docs/erd/*</code> , <code>docs/roadmap/ROADMAP.md</code> , <code>docs/pitch/research-slide.*</code> , <code>docs/adr/*</code>

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of **AEGIS Twin**, a two-scale digital twin of an Indian logistics network. It is the contract between the team and the judges for Levels 2–4. Every requirement has an identifier, a priority, an acceptance criterion and a place in the traceability matrix (§12), so later phases can show exactly what was built and how it was verified.

1.2 Scope

AEGIS Twin models a national supply network (5 ports, 5 plants/suppliers, 6 distribution centres, 12 demand zones and 42 multimodal lanes, all at real coordinates) together with a street-level micro-twin of the **Hyderabad logistics belt** (Medchal ↔ ORR ↔ Shamshabad ↔ Patancheru). The system:

1. **Ingests** high-rate telemetry (GPS, stock counts, supplier ASNs, port status, weather, AIS) through a signed, validated, stream-based pipeline.
2. **Filters** every data point through a 9-layer trust pipeline, so spoofed, corrupted or missing data never corrupts the twin's state.
3. **Mirrors** the network live, using SimPy (macro, discrete-event) and Eclipse SUMO (micro, traffic) co-simulation.
4. **Simulates** disruptions ("what if a cyclone closes Chennai port for 5 days?") with Monte Carlo, then **recommends** ranked mitigation plans that a human can apply.
5. **Proves** its accuracy continuously (the Fidelity Lab) and **scales** as a containerised, observable platform.

Out of scope (WON'T): real IoT hardware, GNN models (future work: SupplyGraph), native mobile apps, and executing real-world transactions such as bookings or payments.

1.3 Definitions, acronyms and abbreviations

Term	Meaning
Twin	The live software model mirroring the network's state
Macro twin	Nation-wide SimPy discrete-event model (nodes, lanes, inventory, orders)
Micro twin	SUMO traffic simulation of the Hyderabad road network
Reality Emulator	A separately seeded coupled simulation with hidden perturbations that <i>plays the physical world</i> and emits device-style telemetry and labelled attacks. It provides ground truth for fidelity scoring.
TTS / TTR	Time-to-Survive / Time-to-Recover (Simchi-Levi). A node is <i>exposed</i> if $TTS < TTR$.
REI	Risk Exposure Index
CRN	Common random numbers: the same seeds across compared plans
CVaR ₉₅	Conditional value-at-risk: mean of the worst 5% of outcomes
MAPE	Mean absolute percentage error
DLQ	Dead-letter queue (here: <code>quarantine</code>)
HMAC	Hash-based message authentication code (SHA-256)
OTIF	On-time in-full
P10/P50/P90	10th/50th/90th percentile
RBAC	Role-based access control
[CP]	On the critical path of the roadmap

1.4 References

1. Team build plan, `PLAN.md` (§1 requirement map, §4 architecture, §6.4 trust layer, §6.5 frontend spec, §7 data model, §10 research).
2. IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications* (structure followed in lightened form).
3. C4 model for software architecture, Simon Brown — used for §4 diagrams.
4. Research and repositories in §11 of this document (the research slide is `docs/pitch/research-slide.png`).

1.5 Overview

§2 describes the product and its users. §3 lists functional requirements (FR-1...FR-25) grouped by the five PNT1 features, followed by non-functional targets. §4–§6 cover architecture, data flow and data model. §7 documents the clickable prototype. §8–§10 cover engine verification, the tech stack, roadmap and risks. §11 grounds the design in research, and §12 traces every requirement to its screen, phase and test.

2. Overall description

2.1 Product perspective

AEGIS Twin is a new, self-contained system. It sits between operational data sources (devices, ERP/WMS, public weather and AIS feeds) and the people who plan and protect the network. Figure 1 shows the system context.

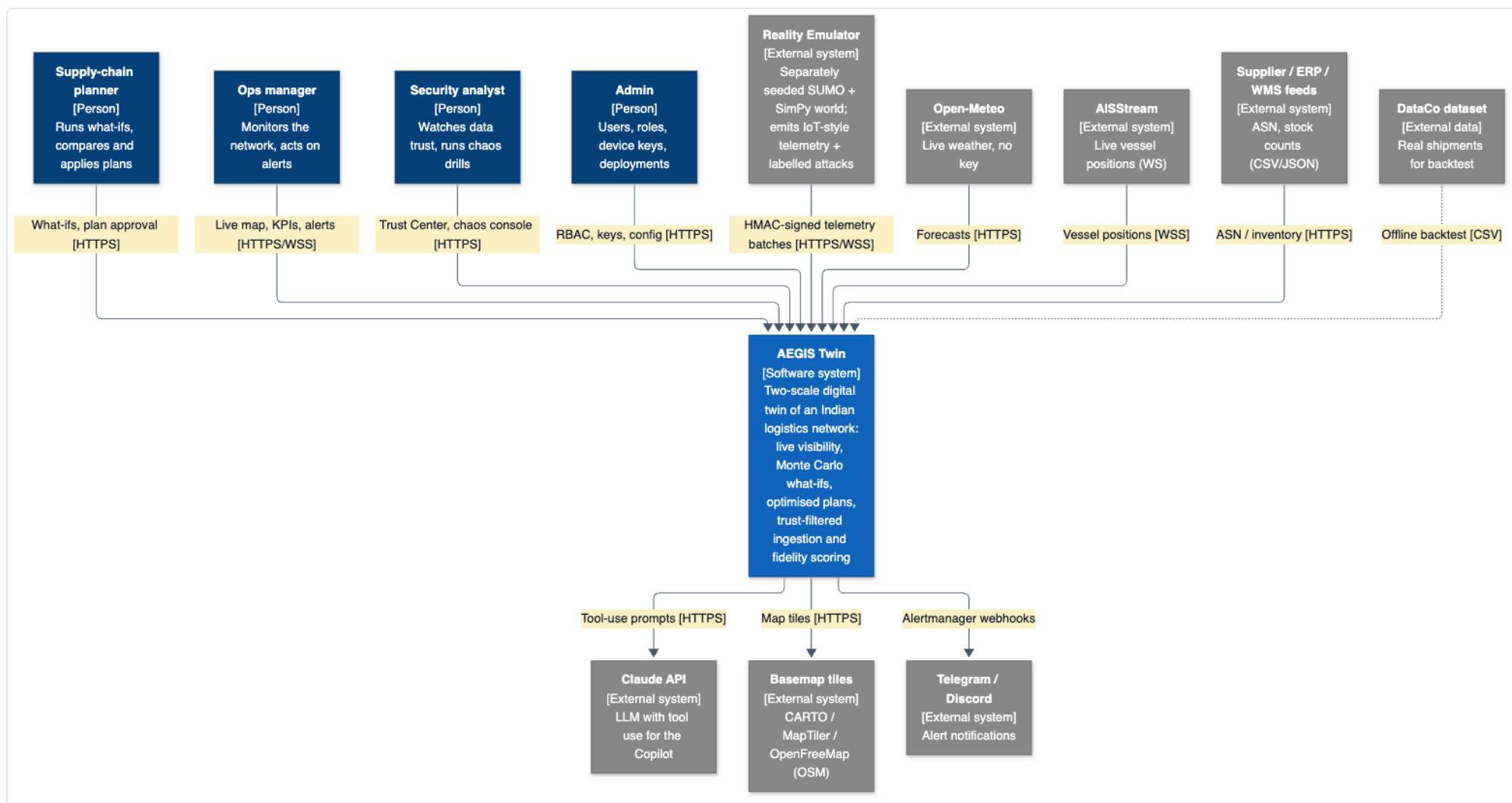


Figure 1 — C4 Level 1: system context

2.2 Product functions (summary)

PNT1 feature	Product function	Screen
F1 Multi-component architecture, high-throughput ingestion, graph modelling	Signed batch ingest → Redis Streams → trust workers → twin state; NetworkX multimodal graph; Timescale + PostGIS; SimPy × SUMO co-simulation	Ops, Control Tower
F2 Scenario-based optimisation, what-if, automated recommendations	Disruption DSL, Monte Carlo, OR-Tools reroute/reallocate, Pareto + CVaR ranking, human-approved Apply	Scenario Lab, Copilot
F3 End-to-end visibility	Live national map, KPIs, alerts, Past/Now/Future timeline, node drill-down, 3D city twin	Control Tower, City Twin
F4 Adversarial resilience	HMAC devices, 9-layer trust pipeline, Kalman + twin-oracle spoof detection, reputation, blackout dead-reckoning, chaos console	Trust Center
F5 Production-ready evaluation	Shadow predictions, MAPE / coverage / F1 / Wasserstein, drift self-recalibration, red-team benchmark, counterfactual decision value, DataCo backtest	Fidelity Lab

2.3 User classes and personas

Persona	Goals	Key tasks	Role (RBAC)	Primary screens
Priya — Supply-chain planner , pharma distributor, Hyderabad	Keep cold-chain vaccines in stock through disruptions at minimum cost	Ask what-if questions; compare plans by cost / service / CO ₂ ; approve a plan	planner	Scenario Lab, Copilot, Control Tower
Arjun — Ops manager , 3PL control room	Know what is late, where, and why, before customers call	Monitor KPIs and alerts; drill into a DC; look 24 h ahead; watch city congestion	viewer (+ planner for apply)	Control Tower, City Twin
Meera — Security analyst	Make sure decisions are never driven by spoofed or corrupted data	Watch source trust; investigate quarantined messages; run red-team drills	security	Trust Center, Fidelity Lab
Admin (platform engineer)	Keep the platform available, secure and observable	Manage users/roles and device keys; deploy; respond to alerts; run chaos drills	admin	Ops, all screens

Judges are a secondary audience: every requirement has a visible on-screen proof (see `PLAN.md` §1).

2.4 Operating environment

- **Client:** evergreen desktop browsers (Chrome/Edge 120+, Firefox 120+, Safari 17+) with WebGL2, at 1366×768 minimum, tuned for 1440×900 and the event's projector.
- **Server:** Linux containers (Docker Compose for development; k3s or managed Kubernetes from Level 4). Python 3.12, PostgreSQL 16 with PostGIS and TimescaleDB, and Redis 7.
- **Frontend hosting:** Vercel (static SPA), with the backend behind Caddy/Ingress with automatic HTTPS.

2.5 Design and implementation constraints

- **C-1** SUMO micro-simulation is limited to a bounded Hyderabad bbox, with motorway→tertiary roads only and ≤ 500 trucks live, so it runs faster than real time (see risk R-1).

- **C-2** A single orchestrator owns the simulation clock (risk R-2).
- **C-3** The LLM Copilot is **evidence-gated**: it can read and propose, but never mutate state. Applying a plan requires a human click by the `planner` role.
- **C-4** Every stochastic component takes an explicit seed, so any result shown on stage can be reproduced.
- **C-5** Every number shown on stage must come from the team's own measured runs. The Level-1 prototype shows **mock data** and says so on every screen.
- **C-6** Free-tier or student-pack resources only: Open-Meteo, AISStream, CARTO/OpenFreeMap tiles, and GitHub Student Pack credits.

2.6 Assumptions and dependencies

- **A-1** No real fleet is available. The Reality Emulator stands in for the physical world, and the pitch says this openly. The ingest API also accepts real feeds (weather, AIS).
- **A-2** Public services (Open-Meteo, AISStream, tile CDNs, the Claude API) may be unavailable during the demo. Circuit breakers and cached fallbacks are required (NFR-6), plus Director mode.
- **A-3** Team of ~4 over ~24 working days (roadmap). Day 1 is taken as 1 Oct 2026 until the level deadlines are confirmed.
- **A-4** The DataCo Smart Supply Chain dataset is used for the real-data backtest only.

3. Specific requirements

Priority uses MoSCoW (`PLAN.md §8`): **M** = must, **S** = should, **C** = could. "Proto" names the prototype screen that already demonstrates the requirement with mock data.

3.1 Functional requirements

F1 — Multi-component architecture, high-throughput ingestion, graph modelling

ID	Requirement (the system shall...)	Pri	Acceptance criterion	Proto
FR-1	...accept batched telemetry, inventory and supplier messages over <code>POST /api/v1/ingest/{telemetry inventory supplier}</code> and <code>WS /ingest/stream</code> , each batch signed with a per-device HMAC-SHA256, and answer <code>202 {accepted, rejected}</code> .	M	Emulator batches of 500 are acknowledged; a tampered batch returns 401 and is logged.	Ops (ingest rate)
FR-2	...validate every message against a strict Pydantic v2 schema (types, ranges, enums) and route failures to <code>quarantine</code> with a reason code, without interrupting processing.	M	A 30% malformed stream yields exact reject counts per reason code and 0 worker crashes.	Trust Center
FR-3	...decouple ingest from processing through Redis Streams consumer groups (<code>telemetry.raw</code> → <code>telemetry.clean</code>) with at-least-once delivery and idempotent de-duplication by message ID.	M	Killing a trust worker mid-stream loses no messages and double-counts none (checked against emulator counts).	Ops (kill pod)

ID	Requirement (the system shall...)	Pri	Acceptance criterion	Proto
FR-4	...maintain a multimodal network graph (nodes; lanes with mode, distance, cost, capacity, CO ₂ /t-km and lognormal lead-time μ , σ) plus live entity state, and persist history in Timescale hypertables.	M	GET /network returns 28 nodes / 42 lanes; GET /nodes/{id} returns state ≤ 1 s old; telemetry history is queryable by vehicle and time.	Control Tower, Network Graph
FR-5	...run a coupled two-scale simulation: SimPy macro twin nation-wide plus SUMO micro twin for Hyderabad. Macro dispatches spawn SUMO trucks, SUMO arrivals fire macro receipts, and SUMO travel-time distributions calibrate macro lanes, all under one orchestrator clock.	S	Integration test: a shipment spawned in macro arrives in micro and is received back in macro, with consistent clocks (± 1 sim-step).	City Twin

F2 — Scenario-based optimisation, what-if analysis, automated recommendations

ID	Requirement (the system shall...)	Pri	Acceptance criterion	Proto
FR-6	...let a planner create a disruption scenario from 7 templates (port closure, cyclone, road flood, demand spike, supplier failure, strike, data blackout) via the scenario DSL {type, target, polygon, start, duration_h, severity, params}, by drag-and-drop onto the map or through the Copilot.	M	Dropping a card on a node creates a valid spec; an invalid spec is rejected with field errors.	Scenario Lab
FR-7	...run Monte Carlo replications of a scenario (configurable N, deterministic seeds, common random numbers), stream progress, and report P10/P50/P90 stock bands, stock-out probability and TTS/TTR per node.	M	500 reps stream progress; re-running with the same seeds reproduces identical bands.	Scenario Lab (fan chart)
FR-8	...generate candidate mitigations: k-shortest multimodal reroutes, min-cost-flow stock reallocation, expedite-by-air and temporary safety-stock buffers.	M	For the Chennai cyclone, ≥ 3 distinct feasible candidates are produced, each with an action list.	Scenario Lab (plans)
FR-9	...evaluate candidates by Monte Carlo (N = 200, CRN), rank them on the cost / service / CO ₂ Pareto front and by a user-weighted score, and show CVaR ₉₅ of lost sales plus a plain-language explanation linked to event-log evidence.	M	Moving weight sliders re-ranks plans; each plan shows KPIs, CVaR ₉₅ and an explanation with evidence references.	Scenario Lab (Pareto, sliders)
FR-10	...apply a plan only after explicit approval by a planner, push the actions into the live twin (routes, transfers, buffers), reroute affected vehicles visibly, write audit_log, and cache scenario results by sha256(spec). The Copilot may propose a plan but never apply one.	M	Apply without the planner role → 403; after Apply, arcs reroute within 2 s, audit row exists; repeated identical what-if returns from cache in < 200 ms.	Scenario Lab, Copilot, City Twin

F3 — End-to-end visibility

ID	Requirement (the system shall...)	Pri	Acceptance criterion	Proto
FR-11	...render a live map of nodes, lanes (flow particles), shipments, ships (AIS), 3D inventory bars, weather and a risk heat-map, each toggleable, with camera presets WORLD → INDIA → HYDERABAD.	M	All 7 layers toggle independently; the preset fly-to completes in 2–3 s.	Control Tower, Intro

ID	Requirement (the system shall...)	Pri	Acceptance criterion	Proto
FR-12	...show at most 4 headline KPIs (fill rate, OTIF, at-risk shipments, ₹ at risk) and a severity-coded alert feed, both updated from the live stream.	M	A state change appears in KPIs and alerts within the NFR-2 latency budget.	Control Tower
FR-13	...provide a Past Now Future timeline scrubber: dragging back replays history, and dragging forward (to +72 h) shows the twin's predicted state as ghosted layers.	M	At +72 h the predicted P50 stock and at-risk nodes are displayed and visually distinct from live data.	Control Tower (timeline)
FR-14	...open a node drill-down with per-SKU inventory vs safety stock, inbound/outbound shipments with ETA P10–P90, and TTS vs TTR with an "exposed" flag.	M	Clicking any DC or port opens the slide-over with all four sections populated.	Control Tower (slide-over)
FR-15	...show the Hyderabad micro-twin in pitched 3D (extruded buildings, SUMO trucks as glowing trails, cars as faint dots), let a user close a road by clicking it, and show rerouting (old path red-dashed, new path green) with the updated corridor travel time.	S	Closing the ORR corridor reroutes affected trucks with <code>rerouteTravelTime</code> and the corridor time updates.	City Twin

F4 — Adversarial resilience

ID	Requirement (the system shall...)	Pri	Acceptance criterion	Proto
FR-16	...authenticate every source with a per-device HMAC-SHA256 key, a timestamp window and nonce replay protection (Redis <code>SETNX</code> + TTL), and support key rotation.	M	Replayed and forged batches are rejected at layer 2/3; rotated keys take effect without restart.	Trust Center
FR-17	...pass every message through the 9-layer trust pipeline (schema, authenticity, temporal, physics, map-matching, Kalman χ^2 gating, twin oracle, feed anomaly, source reputation), and quarantine suspicious data with its reason code and layer, so it never reaches twin state.	M	Each labelled attack type is caught by its intended layer; quarantine entries carry the layer and reason.	Trust Center (pipeline, log)
FR-18	...maintain a Beta-reputation trust score (0–1) per source that down-weights, and then quarantines, persistently bad sources.	M	A source injecting inflated ASNs falls below 0.4 and is quarantined; a clean source stays > 0.85.	Trust Center (source trust)
FR-19	...detect sources silent beyond their SLA, switch the affected entities to <code>PREDICTED</code> dead-reckoning from the simulation, visualise widening uncertainty and a last-seen badge, and fail over demand when a node fails in a cascade.	M	A 30% blackout for 10 min causes no crash and no frozen UI; affected entities show uncertainty cones.	Trust Center (blackout), Network Graph
FR-20	...provide a Chaos Console (<code>security</code> role) that injects labelled attacks (GPS teleport, slow drift, replay, malformed flood, inflated ASN, blackout, cascade failure) and shows detection live, e.g. a spoofed truck as a red ghost linked by a dashed line to its green Kalman estimate.	S	Each injection appears in the quarantine log and on the map within NFR-2 latency; the injection is recorded in <code>attack_labels</code> and <code>audit_log</code> .	Trust Center

F5 — Production-ready evaluation

ID	Requirement (the system shall...)	Pri	Acceptance criterion	Proto
FR-21	...log shadow-mode predictions every N minutes (ETA per active shipment; stock per node/SKU at t+6/12/24 h; stock-out within 24 h), each with P10/P90, and attach the actual outcome when it arrives.	M	The predictions table fills continuously; $\geq 95\%$ of matured predictions get an actual.	Fidelity Lab
FR-22	...compute live fidelity metrics: MAPE/MAE/RMSE, P10–P90 interval coverage with a reliability diagram, stock-out event precision/recall/F1, and Wasserstein distance between simulated and observed travel times per corridor.	M	Metrics recompute on new actuals; the reliability diagram plots nominal vs observed coverage.	Fidelity Lab
FR-23	...raise a drift alarm when rolling MAPE exceeds a threshold, auto-recalibrate lane lead-time parameters by Bayesian update, and surface the "self-healing twin" event in the UI.	C	An injected lead-time drift triggers recalibration, and MAPE returns under threshold within one window.	Fidelity Lab (drift)
FR-24	...run a red-team benchmark of ≥ 500 labelled attacks plus clean traffic and report precision, recall, F1, false-positive rate and mean time-to-detect per attack type, with a confusion matrix.	M	The benchmark is reproducible from its seed; the report matches the ground-truth labels.	Fidelity Lab (confusion)
FR-25	...measure decision value counterfactually (50 random disruptions $\times$ {no action, naïve rule, AEGIS}, CRN) with confidence intervals; backtest the ETA / late-delivery model on held-out DataCo data; and generate a one-click reproducible evaluation report (GET /eval/report).	M	The report lists every metric, seed and engine version, and re-running reproduces it.	Fidelity Lab (decision value, download)

3.2 Non-functional requirements

The six headline NFRs below are **targets**. Each will be validated with the stated method in the stated phase, and only measured values will be presented.

ID	Quality	Target	Validation method	Phase
NFR-1	Ingest throughput	$\geq 5,000$ msgs/s on 4 vCPU	k6 ramp against POST /ingest with emulator-shaped batches; report the max sustained rate at $p95 < 1$ s	P4 (first), P10 (final)
NFR-2	Latency, ingest $\rightarrow$ UI	$p95 < 1$ s	Device timestamp embedded in each message; client measures arrival of the corresponding WS diff; histogram over 10 min	P4, P10
NFR-3	Monte Carlo speed	200 reps $\times$ 30 sim-days < 20 s on 8 cores	pytest-benchmark on the Chennai scenario, multiprocessing.Pool(8), warm cache disabled	P3
NFR-4	UI frame rate	60 fps with 2,000 vehicles	Chrome Performance trace with 2,000 trucks + 3,000 cars on a mid-range laptop and at projector resolution. The prototype already includes a 2k stress toggle and FPS meter (City Twin).	P5, P11
NFR-5	Robustness	0 crashes under 30% malformed or hostile input	Property-based fuzzing (Hypothesis / Schemathesis) plus a 30-minute Chaos Console run at 30% hostile share; count process restarts	P7, P9

ID	Quality	Target	Validation method	Phase
NFR-6	Availability	99.5%	External uptime probe on <code>/healthz</code> over the final week; K8s liveness/readiness probes, PDBs; circuit breakers with cached fallbacks for external APIs	P10
NFR-7	Security	OWASP ASVS L1; JWT access $\leq$ 15 min + refresh; argon2 hashing; TLS everywhere; no secrets in the repo; Trivy/pip-audit/npm audit clean of critical CVEs	CI gates + manual checklist	P8
NFR-8	Reproducibility	Every simulation, benchmark and report is re-creatable from its seed and engine version	Re-run the evaluation report and diff it	P3, P9
NFR-9	Usability	A judge reads each screen's headline in $\leq$ 3 s; $\leq$ 4 KPI cards per screen; no spinner $>$ 1 s during the demo	5-second test with 5 people outside the team	P5, P11
NFR-10	Accessibility	Text contrast $\geq$ 4.5:1 (WCAG AA); every screen keyboard-reachable (1–8, ⌘K); colour never the only carrier of meaning (labels/badges accompany colour)	axe-core + manual keyboard pass	P11
NFR-11	Maintainability	Modular monolith with clear module boundaries; $\geq$ 70% line coverage on <code>sim/</code> , <code>trust/</code> , <code>optimize/</code> ; <code>ruff</code> + <code>mypy</code> + <code>tsc</code> clean	CI	P4–P10
NFR-12	Portability	<code>docker compose up</code> runs the full stack locally; Helm chart for K8s	Fresh-machine test	P4, P10

3.3 Interface requirements

- **User interface:** dark "mission control" visual language (`PLAN.md` §6.5) — background `#070B14`, glass panels, semantic colour only (cyan = healthy, amber = at risk, red = disrupted, violet = AI recommendation, green = recovered), Geist + JetBrains Mono tabular numerals, 200–400 ms transitions. Keyboard: `1–8` screens, `D` demo disruption, `C` chaos, `F` Director mode, `⌘K` palette, `⌘J` Copilot.
- **API:** REST (JSON), WebSockets (msgpack diffs at 5–10 Hz), SSE for Copilot streaming. OpenAPI generated by FastAPI at `/docs`. Endpoints: `/network`, `/nodes/{id}`, `/shipments`, `/kpis`, `/scenarios`, `/scenarios/{id}/optimize`, `/plans/{id}/apply`, `/chaos/inject`, `/trust/*`, `/eval/*`, `/healthz`, `/readyz`, `/metrics`.
- **Software interfaces:** libsumo/TraCI 1.27, SimPy 4.1, OR-Tools, Open-Meteo REST, AISStream WS (server side only), Anthropic Messages API with tool use.
- **Communication:** HTTPS/WSS only (TLS 1.2+), HMAC-signed device batches, CORS restricted to the web origin.

4. Architecture

The system starts as a **modular monolith**: one FastAPI app with clear modules (`ingest`, `trust`, `twin`, `scenarios`, `optimize`, `eval`, `copilot`, `auth`, `chaos`, `ws`), split into containers for Kubernetes in Level 4 (ADR-0001). Figure 2 shows the target containers.

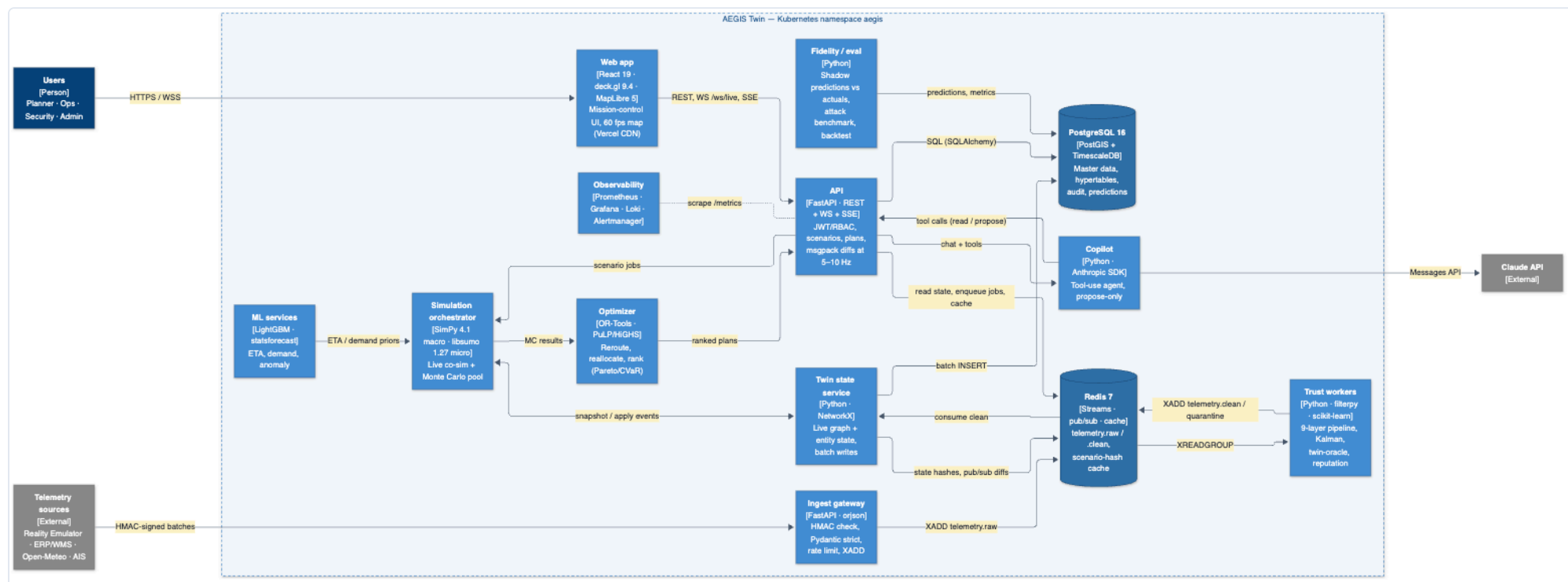


Figure 2 — C4 Level 2: containers

Key design decisions (full ADRs in `docs/adr/`):

1. **ADR-0001 Modular monolith first**, containers later.
2. **ADR-0002 Reality Emulator** as ground truth and a safe place to inject attacks.
3. **ADR-0003 SUMO for fidelity, SimPy for speed**: live mode couples both; Monte Carlo runs macro-only with SUMO-calibrated lead times.
4. **ADR-0004 The twin validates the data**: its predicted state is the reference oracle for spoof detection (trust layer 7).
5. **ADR-0005 Evidence-gated AI**: the LLM proposes and explains; a human applies.

4.1 Sequence (a) — telemetry ingest

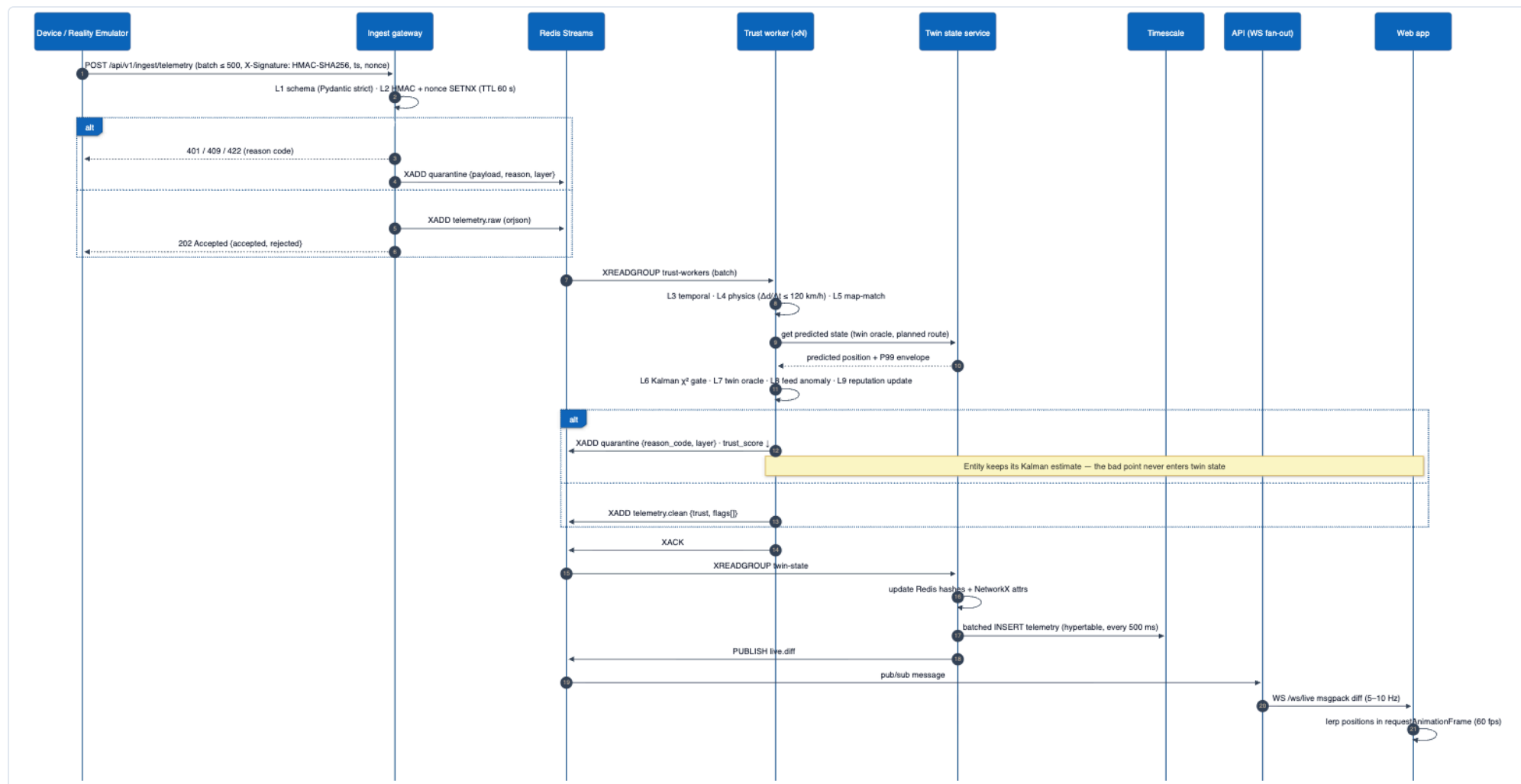


Figure 3 — Sequence: telemetry ingest

4.2 Sequence (b) — what-if scenario

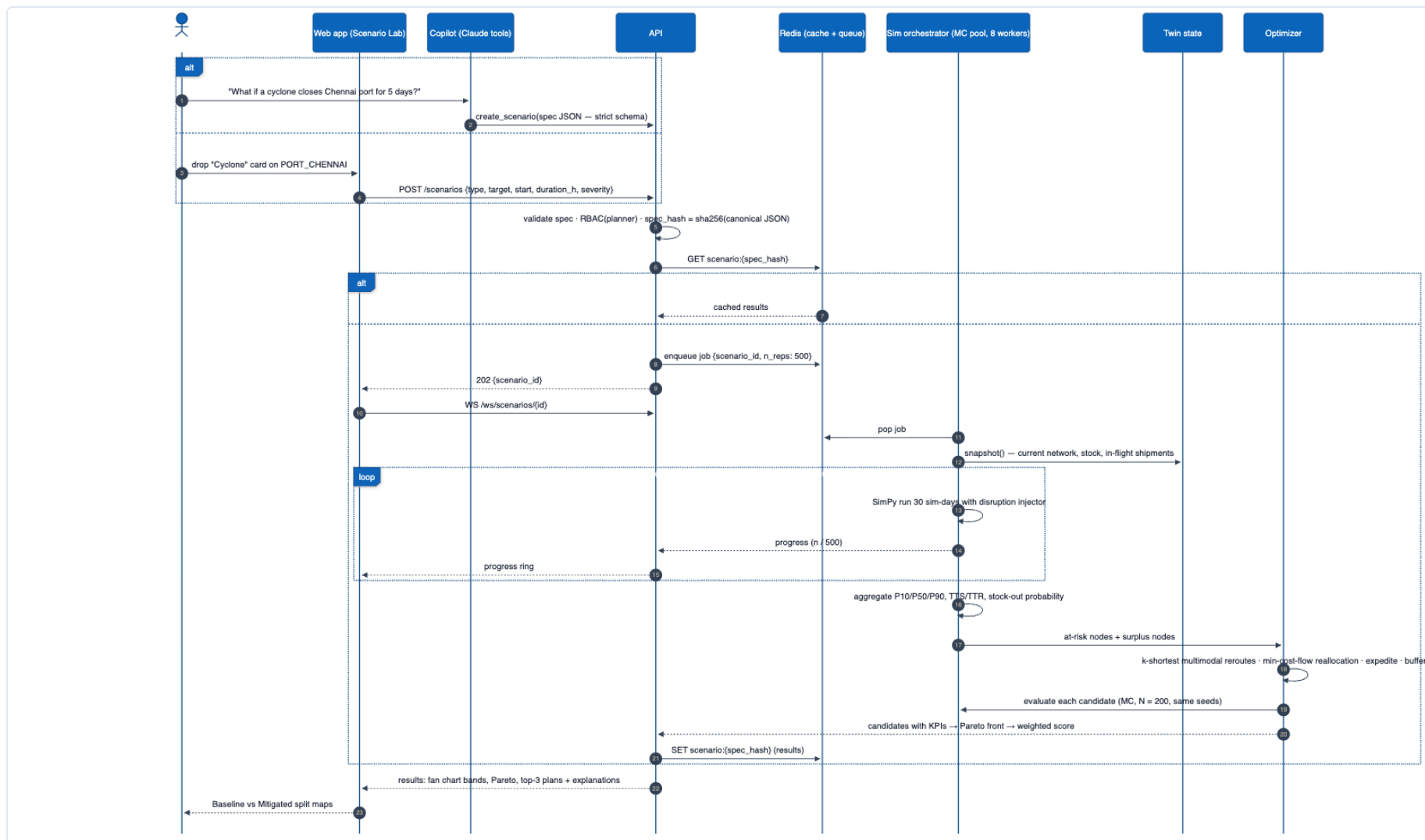


Figure 4 — Sequence: what-if scenario

4.3 Sequence (c) — apply plan

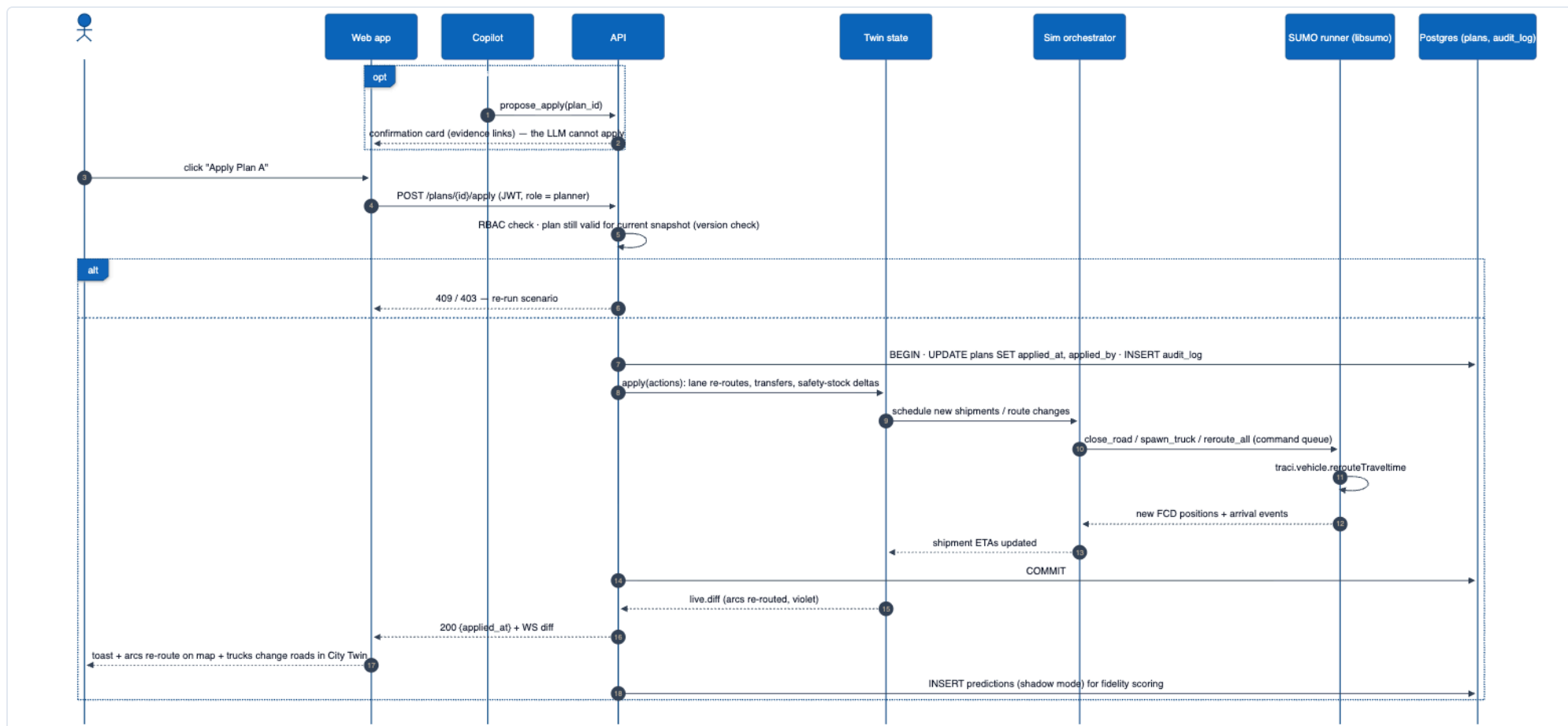


Figure 5 — Sequence: apply plan

5. Data flow

5.1 DFD Level 0 — system context

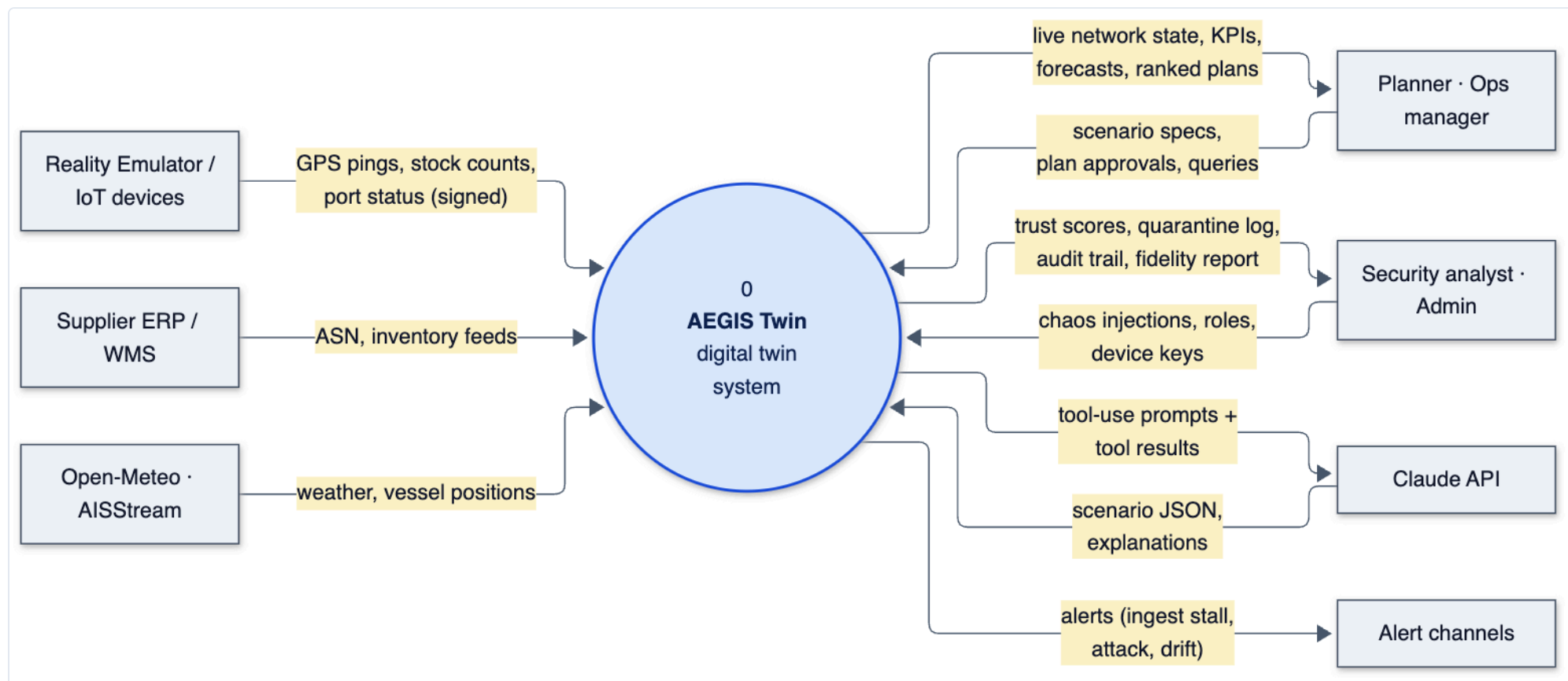


Figure 6 — DFD Level 0

5.2 DFD Level 1 — ingest → trust → twin → simulate/optimise → UI

Processes are circles; data stores are cylinders (D1–D7). Bad data leaves the main flow at 1.0 (schema/auth) and 2.0 (trust) into D2 and never reaches D4. The twin (3.0) feeds its own predictions back into 2.0 as the spoofing oracle.

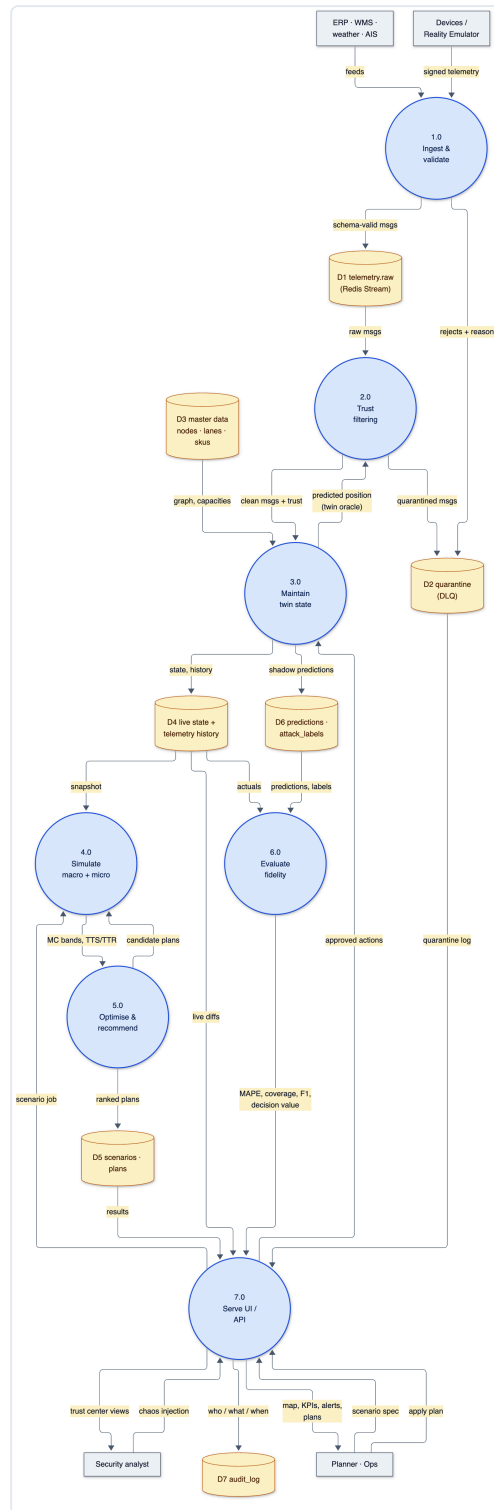


Figure 7 — DFD Level 1

6. Data model

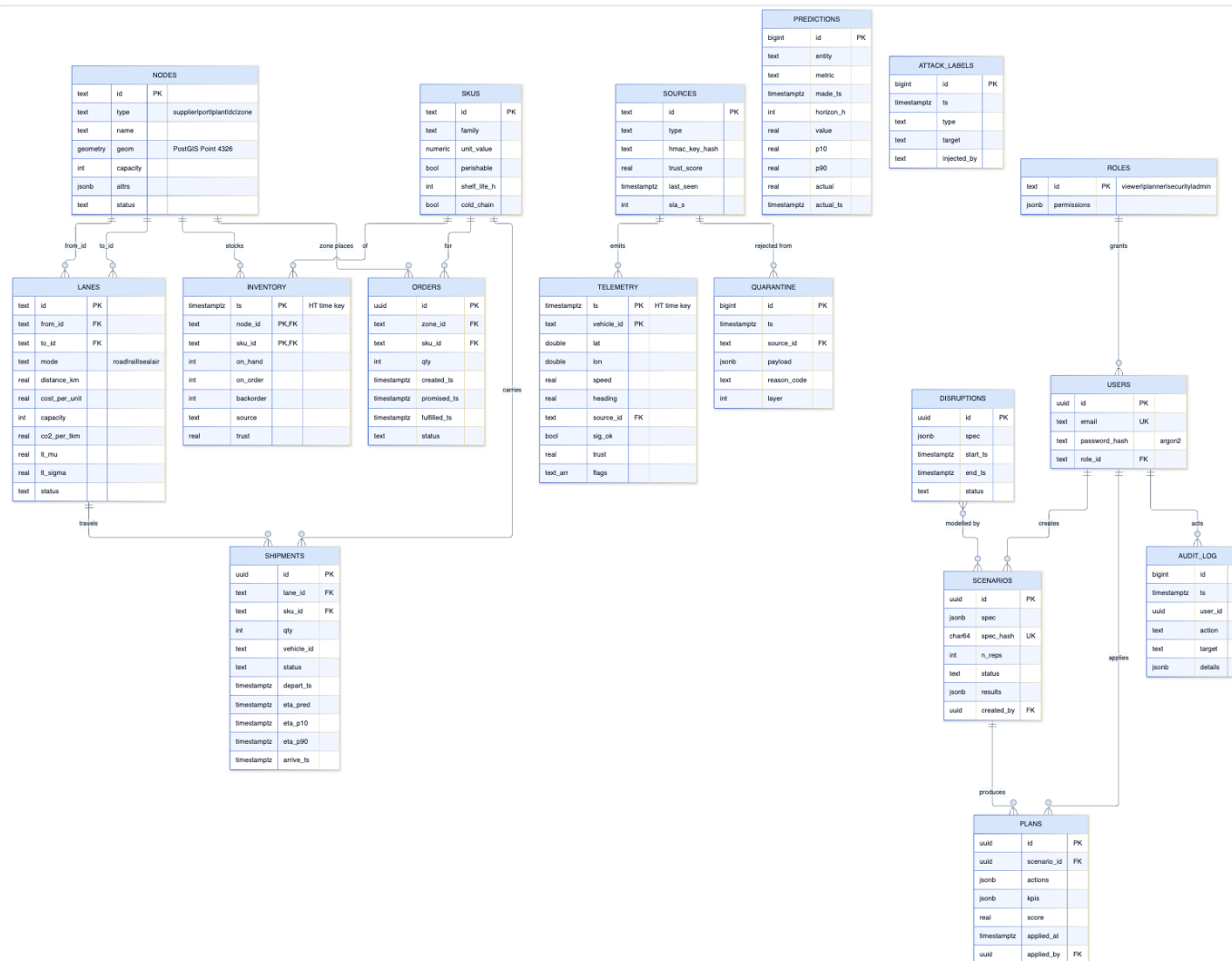


Figure 8 — Entity-relationship diagram

Table	Purpose	Notes
nodes , lanes , skus	Master data: the network graph	geom PostGIS point (GiST index); lanes carry lognormal lead-time μ , σ
inventory	Stock per node $\times$ SKU over time	Timescale hypertable; source and trust recorded per reading
orders , shipments	Demand and flows	Shipments carry ETA P50 plus P10/P90
telemetry	Vehicle positions	Hypertable; compression policy after 1 day; index (vehicle_id, ts DESC) ; sig_ok, trust, flags[]
sources , quarantine	Trust layer state and DLQ	Only the key hash is stored; quarantine keeps the raw payload, reason code and layer
disruptions , scenarios , plans	What-if lifecycle	spec_hash unique $\rightarrow$ cache key; applied_by / applied_at for accountability
predictions , attack_labels	Evaluation ground truth	Feeds the Fidelity Lab and red-team benchmark
audit_log , users , roles	Security	Every apply, chaos injection and login is audited

Retention: raw telemetry is kept 7 days uncompressed, then compressed; continuous aggregates hold KPI history at 1-minute resolution. Quarantine and audit logs are kept for the whole event.

7. User interface — clickable prototype

The Level-1 prototype is **built in code, not in a drawing tool**, so it becomes the real frontend skeleton (apps/web: Vite + React 19 + TypeScript + Tailwind v4 + deck.gl 9.4 + MapLibre 5 + ECharts). It covers all nine §6.5 screens with static mock JSON produced by scripts/gen_mock.py (seeded). Mock values include a toy Monte Carlo whose baseline stock-out time of 3.2 days emerges from the model rather than being typed in. Real geography: node coordinates are real, and Hyderabad road geometry comes from OpenStreetMap via OSRM.

Honesty rule (C-5): every screen carries a *PROTOTYPE · MOCK DATA* badge. None of these numbers will be quoted as results.

#	Screen	Demonstrates	Figure
1	Intro	Globe $\rightarrow$ India $\rightarrow$ arcs draw in $\rightarrow$ tagline (skippable)	9
2	Control Tower	Map layers, KPIs, alerts, timeline, node slide-over, ₹K	10–12
3	Scenario Lab	Drag disruption $\rightarrow$ run 500 reps $\rightarrow$ split Baseline/Mitigated maps, fan chart, Pareto, plans $\rightarrow$ Apply	13–14
4	City Twin	3D Hyderabad, trucks on real roads, flood a corridor $\rightarrow$ reroute	15
5	Trust Center	Chaos Console, 9-layer rejects, ghost vs Kalman, blackout cones, quarantine log	16
6	Fidelity Lab	ETA vs actual, reliability diagram, confusion matrix, decision value, drift	17
7	Network Graph	3D force graph, cascade animation, REI ranking	18

#	Screen	Demonstrates	Figure
8	Ops	Grafana-style panels, pod table with kill → self-heal, k6 summary	19
9	Copilot drawer	Streaming chat with tool-call cards; proposal requires human Apply	20

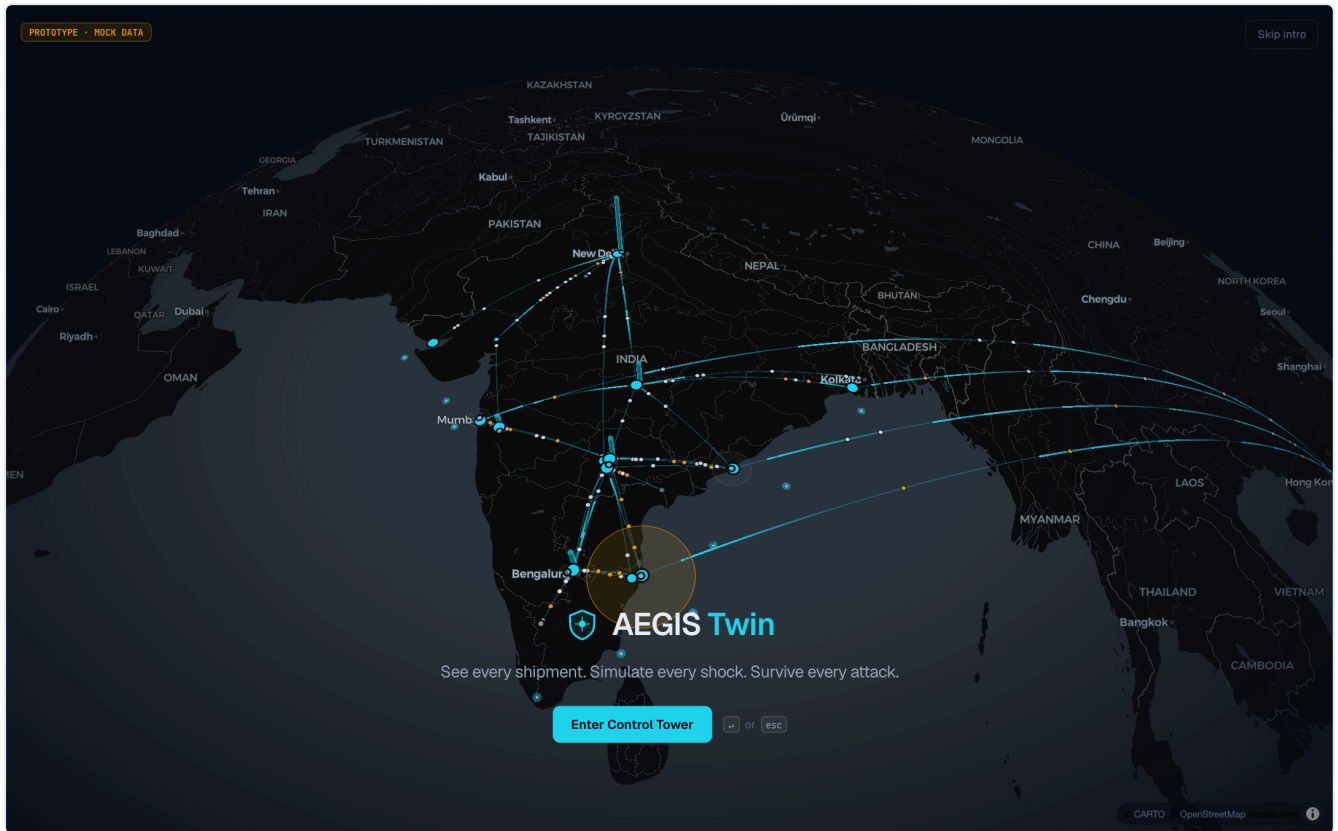


Figure 9 — Intro

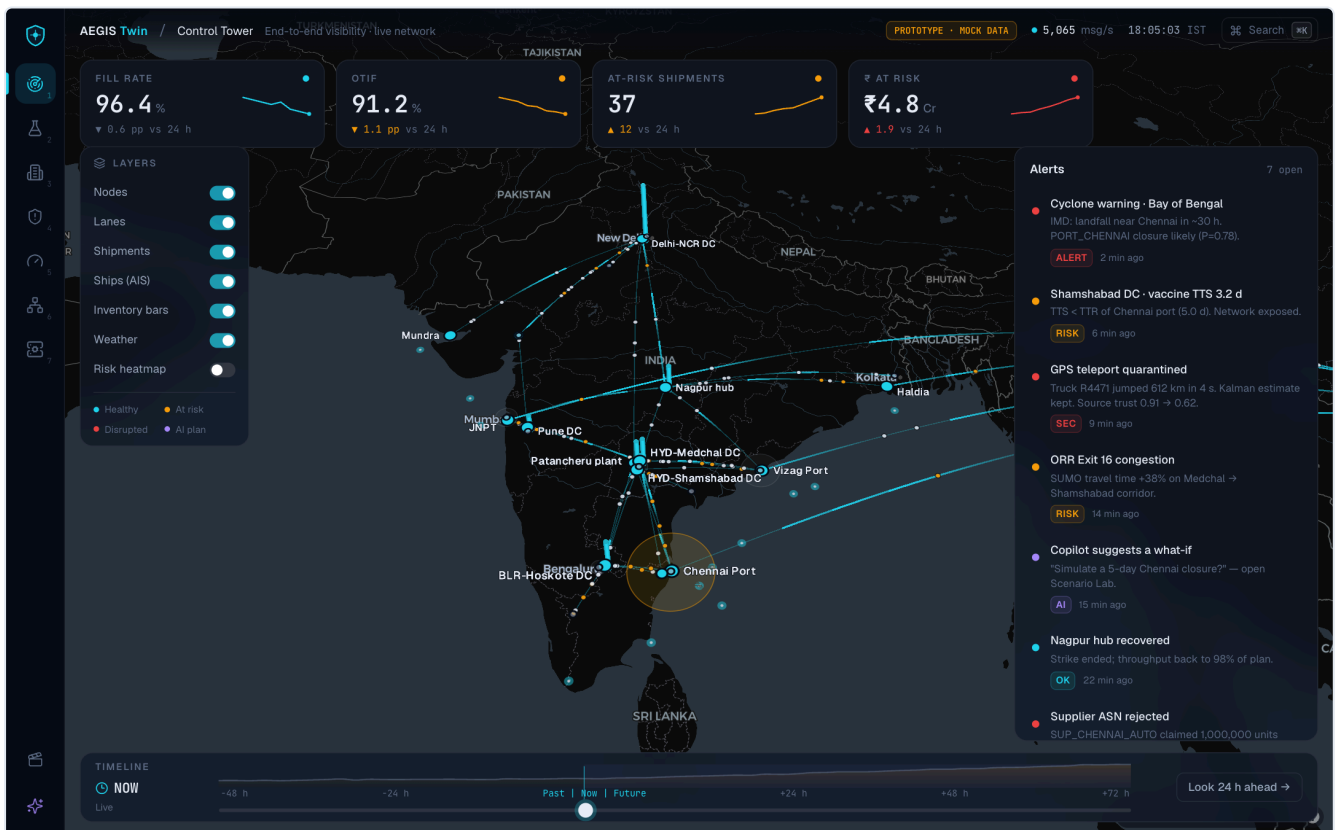


Figure 10 — Control Tower (home)

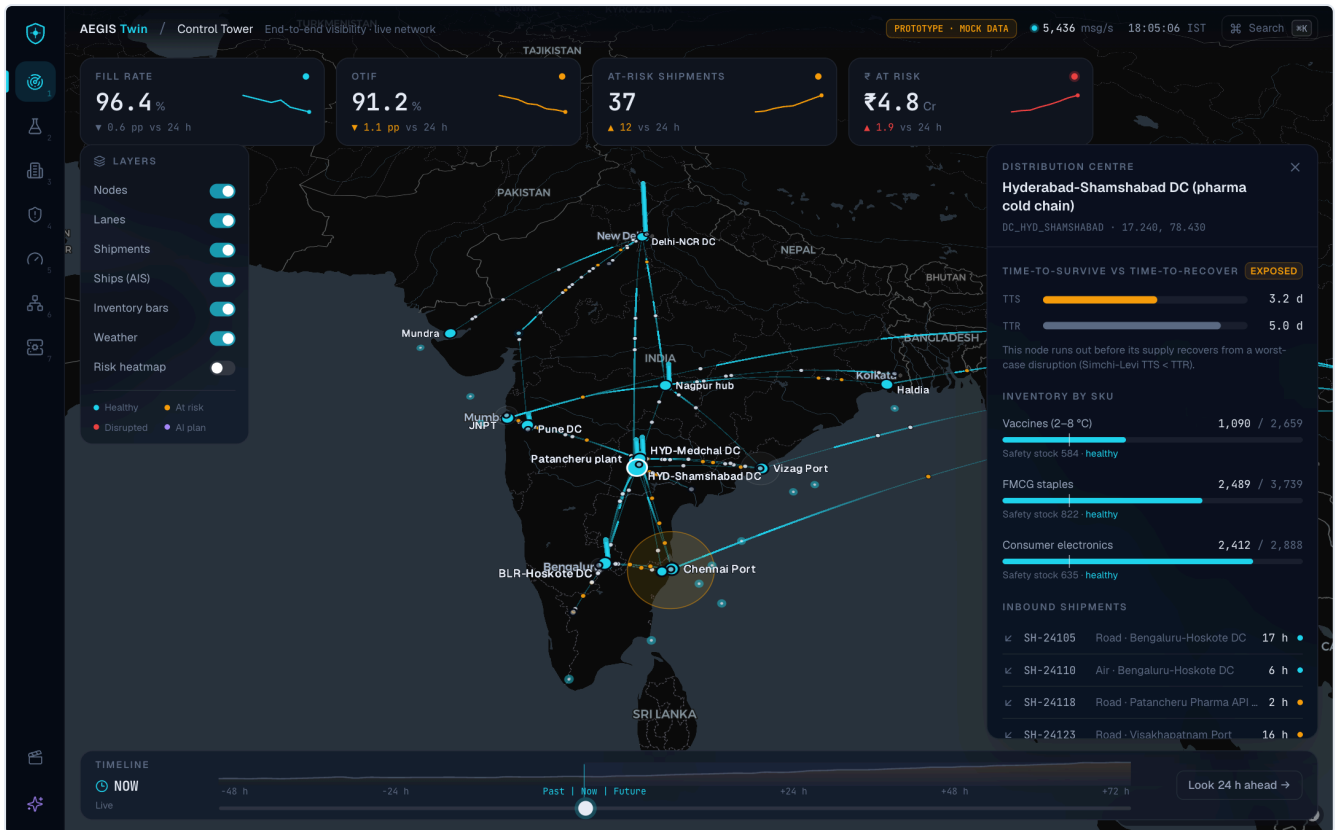


Figure 11 — Control Tower: node drill-down (TTS vs TTR, inventory, shipments)

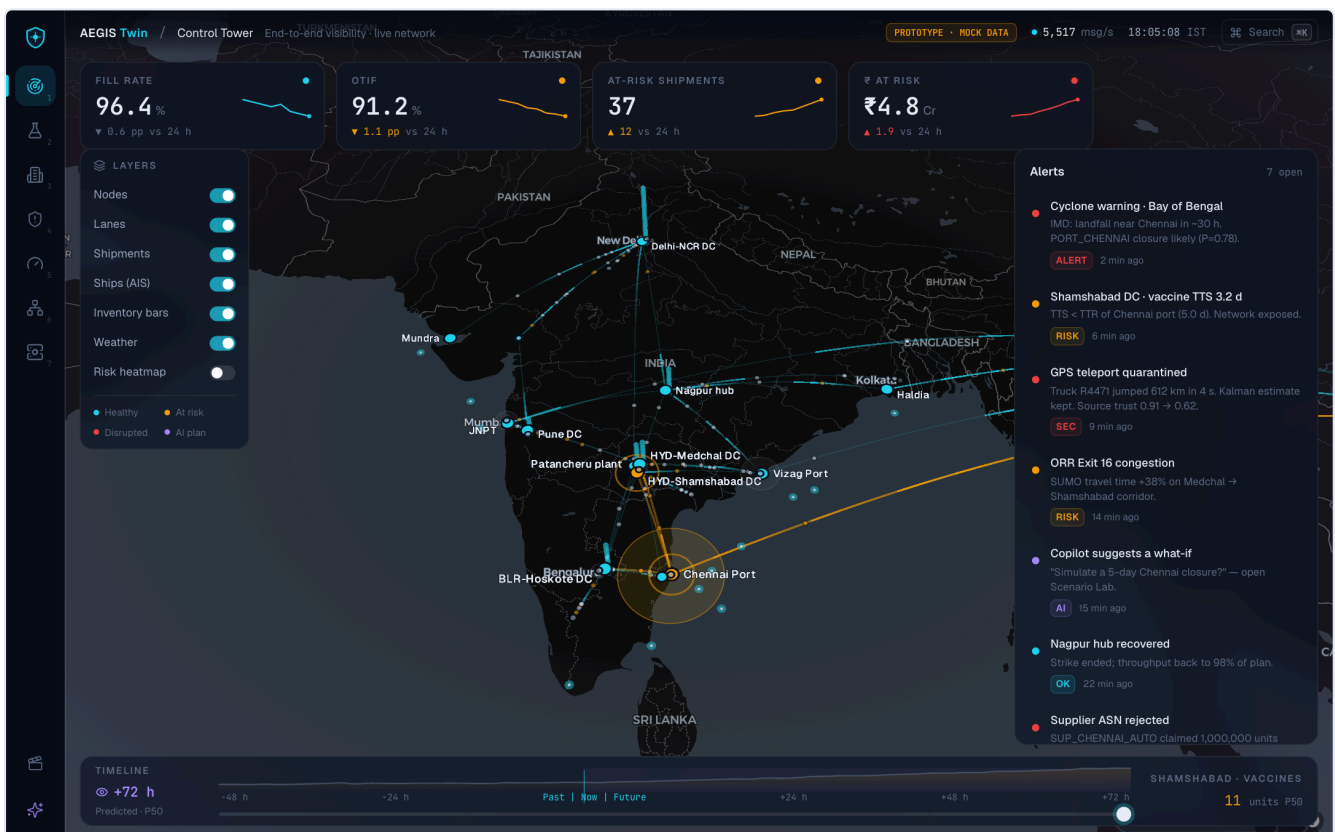


Figure 12 — Control Tower: timeline dragged to +72 h (predicted, ghosted)

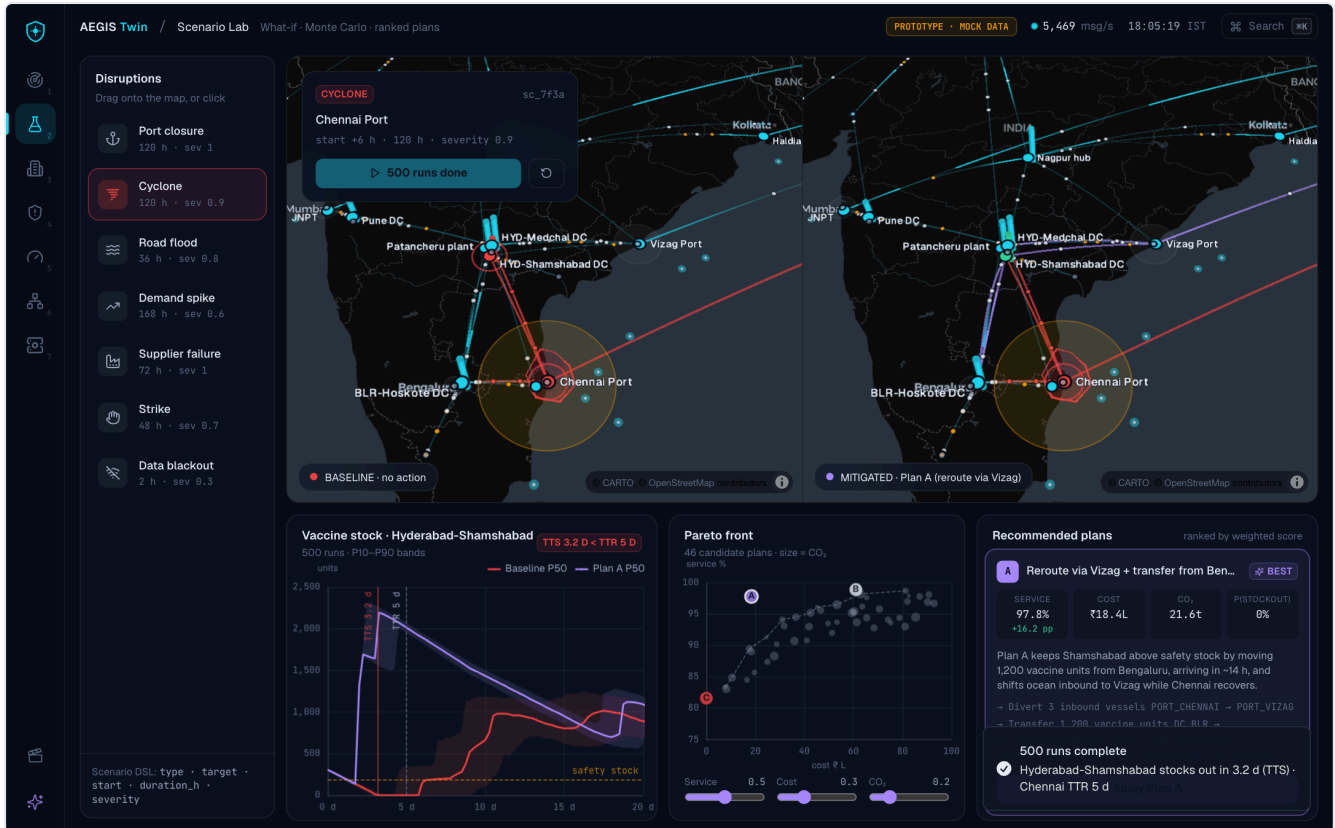


Figure 13 — Scenario Lab: 500-run Monte Carlo results

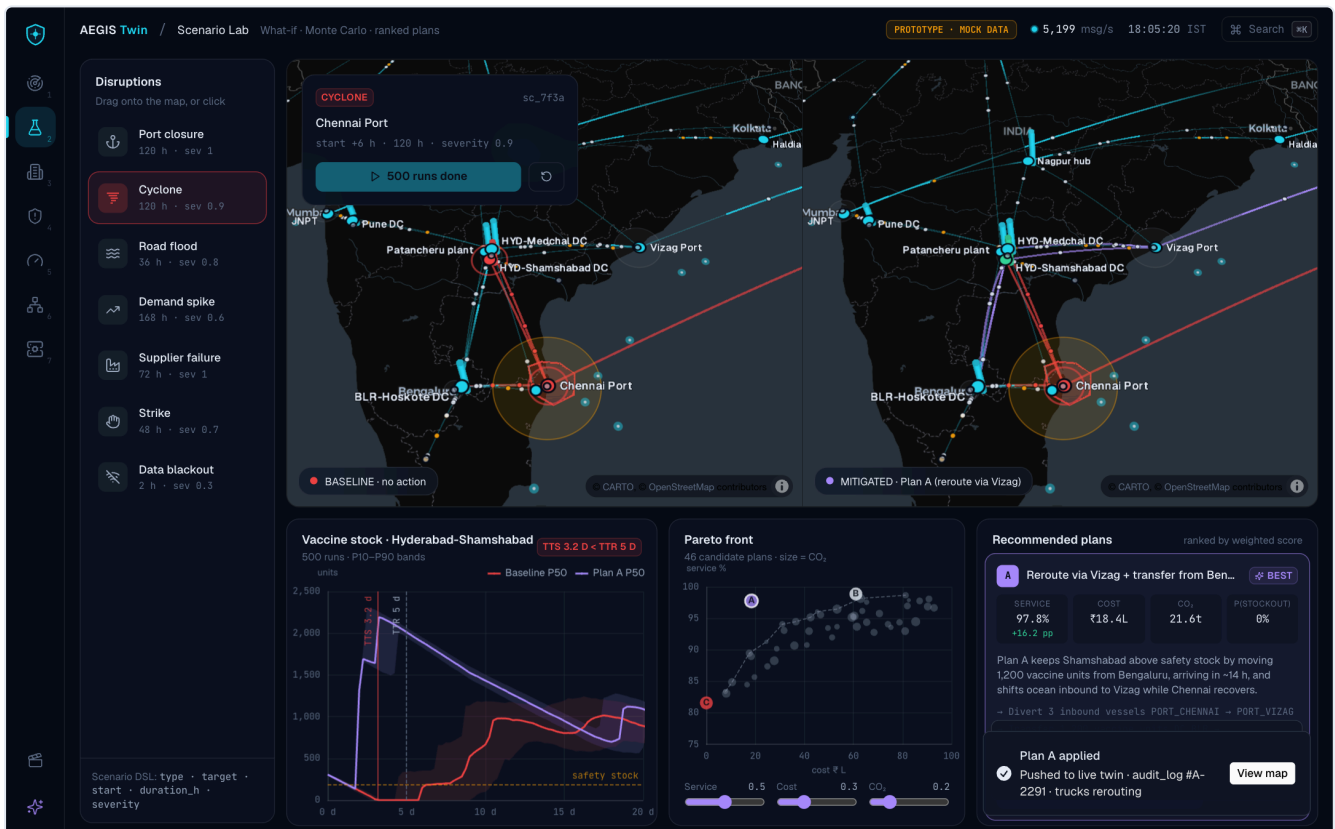


Figure 14 — Scenario Lab: Plan A applied (human approval)

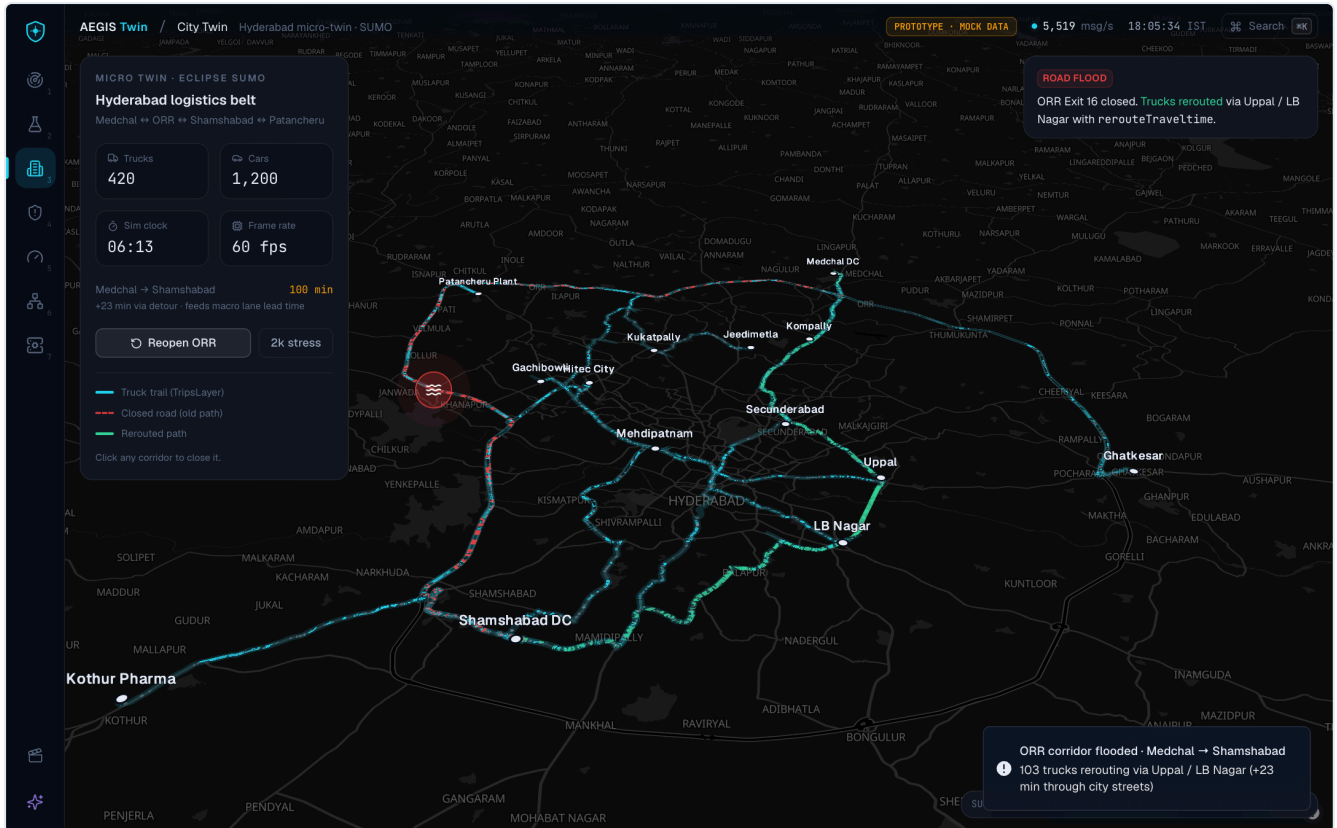


Figure 15 — City Twin: ORR flooded, trucks rerouted

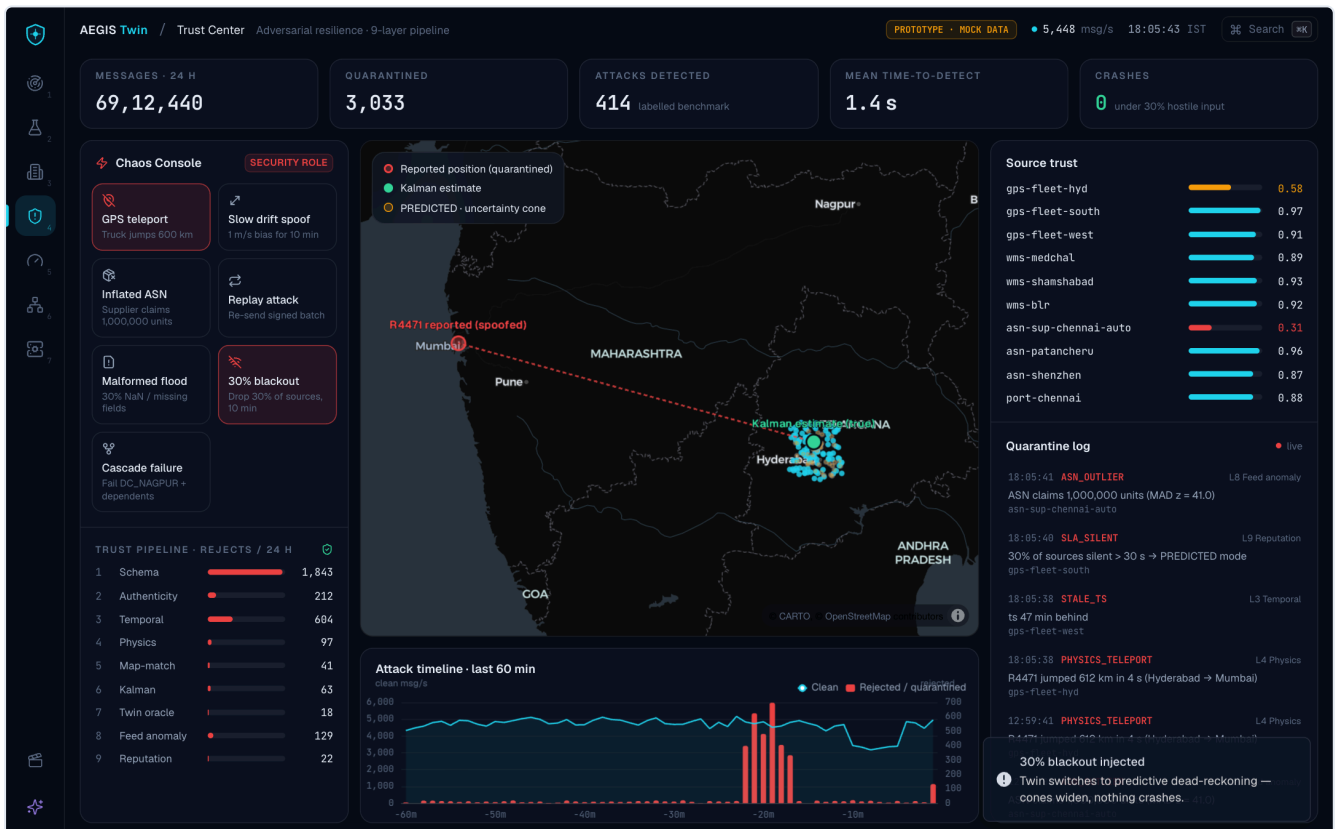


Figure 16 — Trust Center: GPS teleport caught + 30% blackout

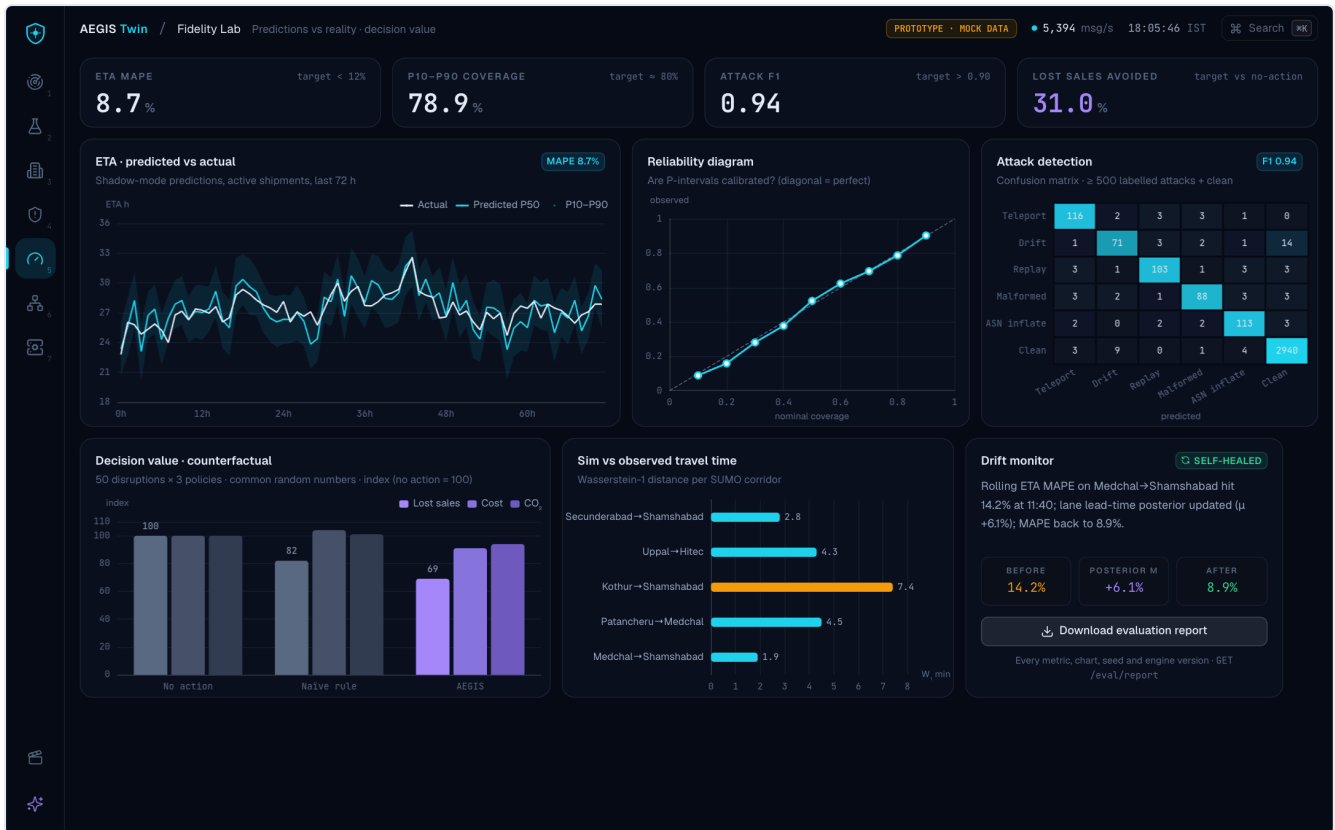


Figure 17 — Fidelity Lab

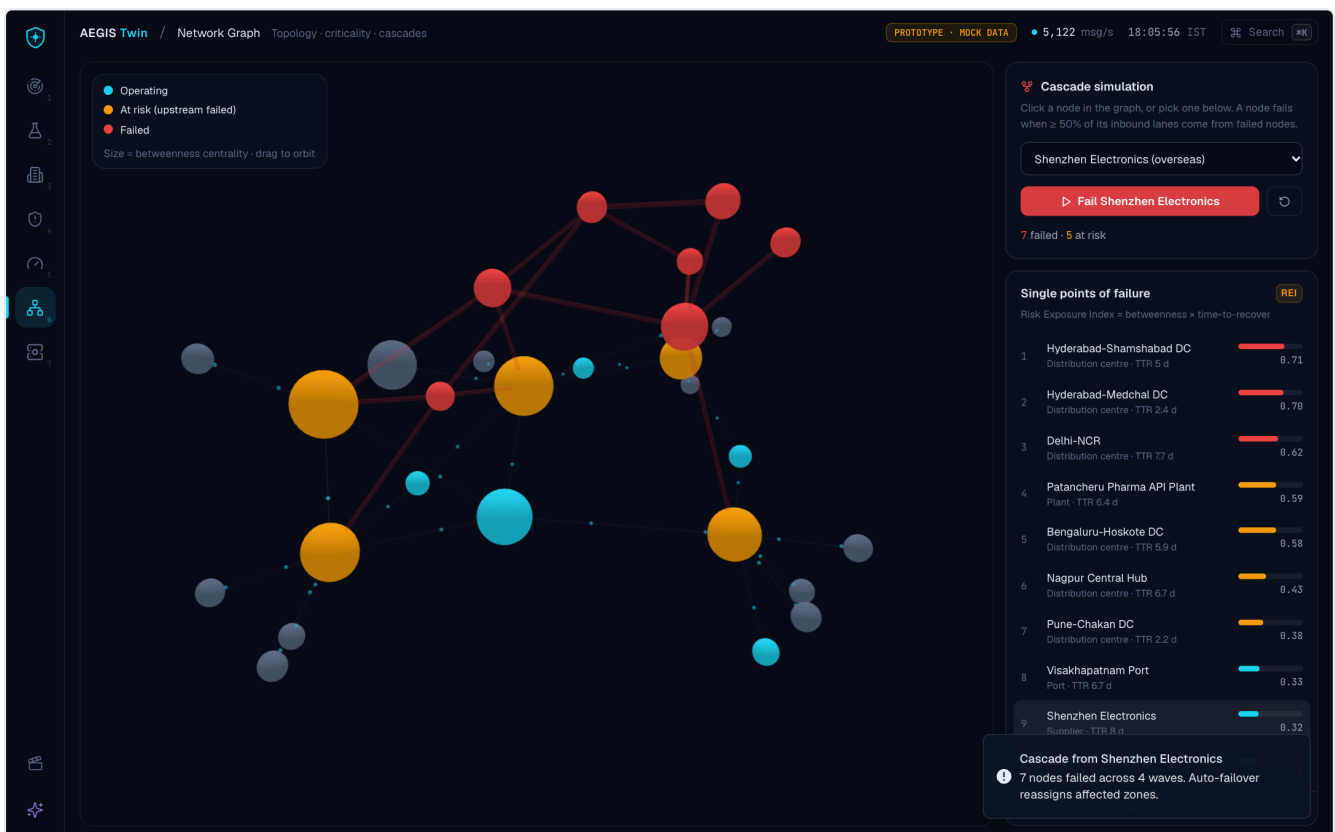


Figure 18 — Network Graph: cascade from an overseas supplier

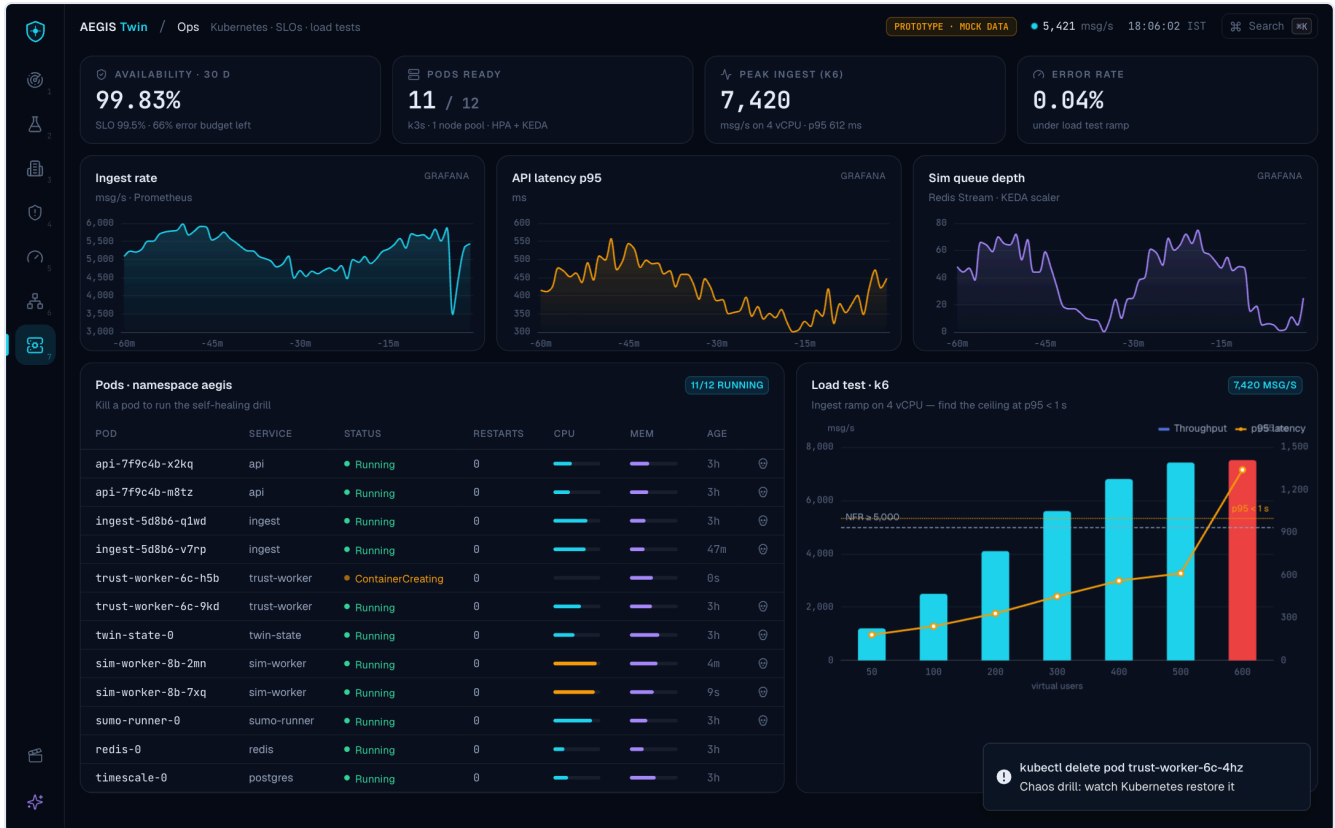


Figure 19 — Ops: pod killed, Kubernetes self-heals

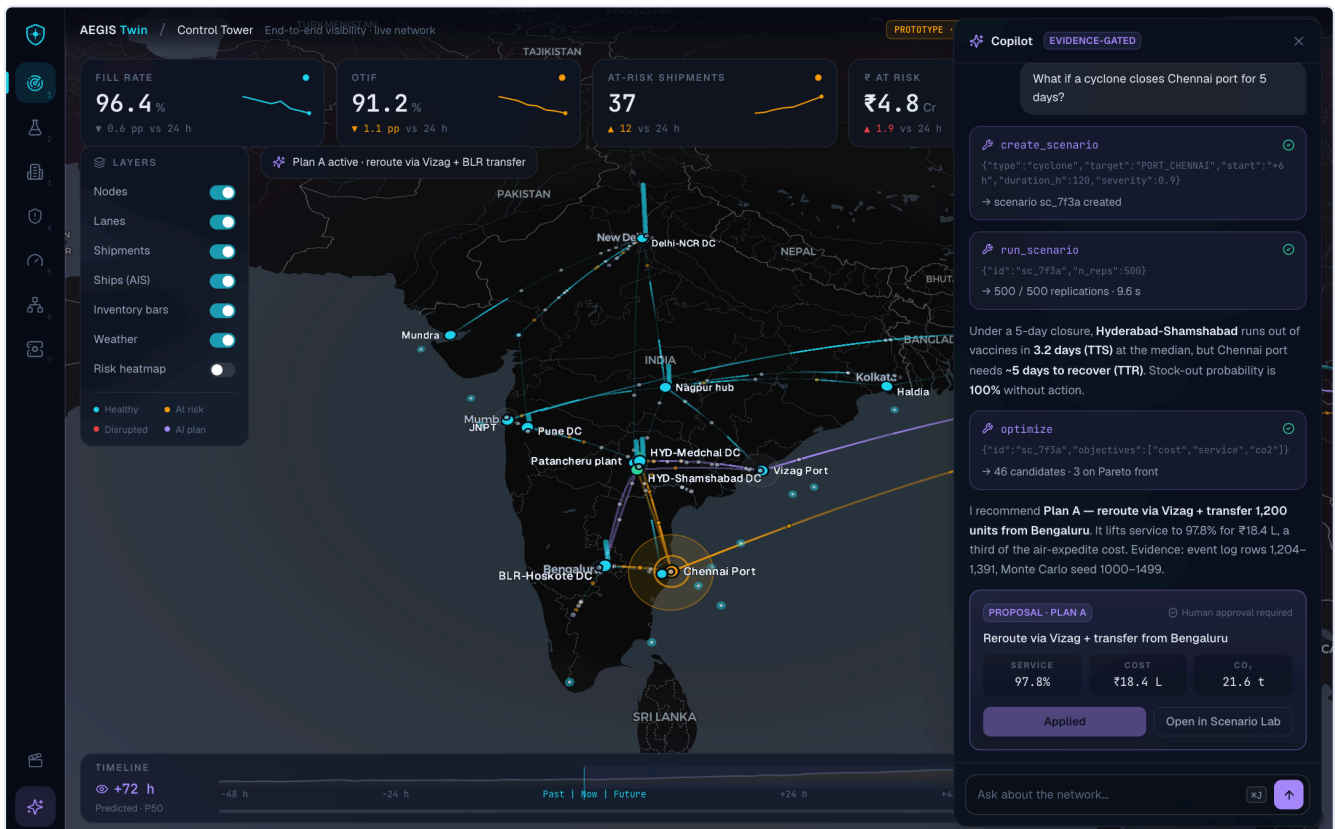


Figure 20 — Copilot: evidence-gated proposal

All 20 captures are in `docs/wireframes/` and are regenerated by `tools/screenshots.mjs`, which drives the prototype like a user and doubles as an end-to-end smoke test.

8. Engine verification

Before any twin number is trusted, the engines are checked against known answers. The checks run with `python -m sim.macro.verify` (report: `docs/sim/engine_verification.json`) and in the test suite. Results measured on 30 Sep 2026 (engine `macro-3.0`):

#	Check	Set-up	Measured	Pass
V1	Single-node (s,S), Poisson demand vs the analytic fill rate	$\lambda = 20/\text{day}$, $L = 2 \text{ d}$, $(s,S) = (45, 80)$, daily review, backorders, 20,000 days. Analytic: Markov chain of the post-order inventory position Y ; fill = $E[\min(D, (Y - D_L)^*)]/\lambda$	86.94% simulated vs 86.97% analytic (0.03% rel. error)	✓ ≤ 2%
V2	EOQ sanity check	deterministic $D = 240/\text{day}$, $K = ₹5,000$, $h = ₹1/\text{unit/day}$, (R,Q) continuous review; Q from 600 to 3,000 in steps of 150, each over 200 full cycles	cost minimum at $Q = 1,500$, the grid point nearest EOQ = 1,549; the curve matches $K \cdot D/Q + h \cdot (Q/2 + SS)$ exactly	✓
V3	3-node toy cross-checked against SupplyNetPy 0.1.12	infinite supplier → distributor $(s,S) = (40, 90)$ → Poisson unit demand, $L = 48 \text{ h}$, lost sales; 20 seeds × 20,000 h in each engine	fill 85.80% vs 85.81% ($\Delta 0.01 \text{ pp}$); on-hand $\Delta 0.3\%$; orders $\Delta 0.2\%$	✓
V4	Macro ↔ micro handshake (FR-5)	coupled run, 30 sim-h, SUMO lock-step every 60 s	every shipment handed to SUMO is received back in macro; SUMO spawn time equals macro time; receipt lag ≤ 1 sync step (max 50 s)	✓
V5	SUMO calibration	84 trucks on the five in-city corridors; Wasserstein-1 between the macro prior and the SUMO drive times	$W1 = 4\text{--}62 \text{ min per lane}$; calibrated lanes stored in <code>sim/micro/hyderabad/calibration.json</code>	reported

V1 caught a real defect: a receipt and a review in the same simulated instant counted the delivery twice. It was fixed before any KPI was reported. NFR-3 was also measured in Phase 3: 200 Monte Carlo replications × 30 days took **2.8 s** on 8 cores, against a target of < 20 s. Details, including the macro stress test (TTS/TTR per node), are in `docs/sim/PHASE3.md`.

9. Technology stack

Layer	Choice	Why
Macro DES	SimPy 4.1 (custom engine), <code>simpy.rt.RealtimeEnvironment</code>	Full control over live hooks; wall-clock sync
Micro traffic	Eclipse SUMO 1.27 via libsumo (TraCI for debugging)	In-process speed; real OSM roads; rerouting
Graph / optimisation	NetworkX; OR-Tools min-cost-flow and VRP; PuLP + HiGHS	k-shortest paths, centrality, cascades; exact LPs

Layer	Choice	Why
ML	LightGBM (ETA), statsforecast (demand), scikit-learn IsolationForest, filterpy (Kalman)	Small, fast, explainable
AI Copilot	Claude API with tool use (Sonnet for speed, Opus for hard reasoning)	Natural language → strict scenario JSON; evidence-gated
API	FastAPI, Pydantic v2 strict, uvicorn, orjson, msgpack	Async REST + WS + SSE; fast validation
Bus / cache	Redis 7 Streams, pub/sub, spec-hash cache	Consumer groups, backpressure, instant repeat what-ifs
Database	PostgreSQL 16 + PostGIS + TimescaleDB	Geo + time series in one engine
Frontend	Vite, React 19, TypeScript, Tailwind v4, deck.gl 9.4, MapLibre 5, ECharts 6, Motion, Zustand, cmdk, sonner	60 fps WebGL maps, globe view, rich charts
DevOps	GitHub Actions, Docker, k3s + Helm, HPA/KEDA, Caddy, Prometheus, Grafana, Loki, Alertmanager, k6/Locust, Vercel	Level 3–4 enterprise readiness on student credits

10. Roadmap and risks

The dated Gantt for Phases 2–11, per-phase exit criteria and the full risk register are in `docs/roadmap/ROADMAP.md`. Summary:

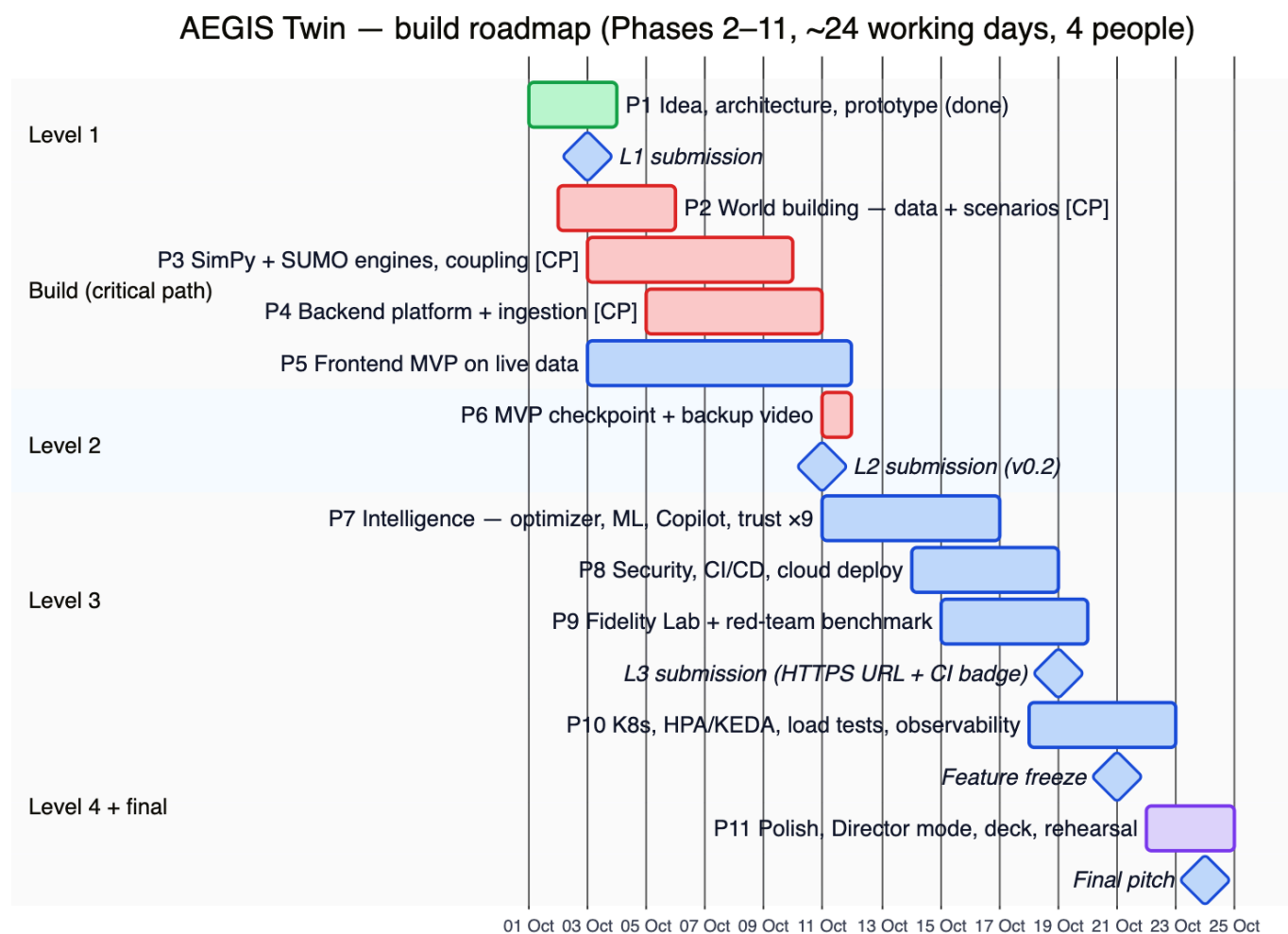


Figure 21 — Roadmap, Phases 2–11

ID	Risk	L	I	Mitigation
R-1	SUMO network too large or slow	M	H	Small bbox, road-type filter, libsumo (not TraCI), cap 500 trucks, low-density background traffic; benchmark on Day 3
R-2	Macro/micro time-sync bugs	M	H	One orchestrator owns the clock; handshake integration test in CI (FR-5)
R-3	Live demo fails (Wi-Fi, cloud, API)	M	H	Director mode runs fully local from a recorded seed; backup video; cached tiles; phone hotspot
R-4	Scope creep	H	M	MoSCoW cut list; feature freeze 3 days before the final
R-5	"Is this real?" scepticism	M	M	State the Reality Emulator openly; show real OSM, weather, AIS and DataCo integrations; quote measured numbers only
R-6	LLM latency or cost in the demo	M	M	Cache the scripted question; fast model for tool routing
R-7	Projector washes out the dark UI	M	M	Test early at projector resolution; high-contrast toggle

11. Research grounding

Source	What we took	Where it shows up
Eclipse SUMO docs + <i>A SUMO-Based Digital Twin for Evaluation of Conventional and EV Networks</i> (arXiv 2507.10280)	SUMO as a credible twin substrate, validated by real-vs-sim accuracy checks	Micro twin (FR-5, FR-15); Wasserstein corridor fidelity (FR-22)
sidewalklabs/sumo-web3d	TraCI → WebSocket frame-diff → browser pattern	msgpack diffs at 5–10 Hz with client-side lerp to 60 fps (NFR-2, NFR-4)
SupplyNetPy (arXiv 2607.09745, GitHub)	SimPy supply-network patterns ((s,S), (R,Q), disruptions) validated against AnyLogistix, EOQ and newsvendor	Macro engine design and engine verification (§8)
SimPy real-time env + dynamic-des	Wall-clock DES with live parameter updates	Live-sync twin; parameter switchboard for scenario injection
Ivanov et al. , <i>Digital Supply Chain Twins: Managing the Ripple Effect</i>	Ripple effect as the conceptual foundation	Cascade simulation and Network Graph (FR-19)
Simchi-Levi , TTS / TTR / REI	Time-to-survive vs time-to-recover exposure analysis	Node drill-down, Scenario Lab headline "TTS 3.2 d < TTR 5 d", REI ranking (FR-7, FR-14)
GPS-IDS (arXiv 2405.08359) + decomposition Kalman filter under GPS spoofing (Sci. Rep.)	Physics model + estimator-based spoof detection	Trust layers 4 and 6: speed/teleport checks, Kalman χ^2 gating (FR-17)
Disruption Detection for a Cognitive Digital Supply Chain Twin (arXiv 2309.14557)	Anomaly detection on supply feeds	Trust layer 8 feed-anomaly (IsolationForest + MAD; autoencoder as an upgrade)
Digital Twin Counterfactual Framework (arXiv 2604.01325)	Fidelity hierarchy for validating simulated outcomes	Fidelity Lab: shadow predictions, counterfactual decision value (FR-21–FR-25)

Source	What we took	Where it shows up
SupplyTwin-Simulation	Validation-first ingestion, deterministic seeds, evidence-gated AI	C-3, C-4, FR-2, FR-10
OR-Tools min-cost flow	Exact network-flow reallocation	FR-8
deck.gl 9.4 + MapLibre (globe, TripsLayer)	Interleaved WebGL layers on a globe	Intro, Control Tower, City Twin

The one-page research slide is `docs/pitch/research-slide.png`. Future work: the **SupplyGraph** GNN benchmark (arXiv 2401.15299).

12. Traceability matrix

FR	Feature	Persona	Prototype screen	Build phase	Verification
FR-1	F1	Admin	Ops	P4	k6 + signature tests
FR-2	F1	Security	Trust Center	P4	Fuzz (Hypothesis), NFR-5
FR-3	F1	Admin	Ops (kill pod)	P4	Chaos drill: count parity
FR-4	F1	Ops	Control Tower, Network Graph	P2, P4	API contract tests
FR-5	F1	Ops	City Twin	P3	Macro↔micro handshake test (§8 V4 — passing)
FR-6	F2	Planner	Scenario Lab	P2, P5	DSL schema tests; Playwright drag-drop
FR-7	F2	Planner	Scenario Lab	P3	Seeded reproducibility test; NFR-3 benchmark (2.8 s — passing)
FR-8	F2	Planner	Scenario Lab	P7	Unit tests on toy graphs
FR-9	F2	Planner	Scenario Lab	P7	Pareto dominance property test
FR-10	F2	Planner	Scenario Lab, Copilot, City Twin	P7, P8	RBAC 403 test; audit row; cache timing
FR-11	F3	Ops	Control Tower, Intro	P5	Playwright visual smoke; NFR-4
FR-12	F3	Ops	Control Tower	P5	NFR-2 latency probe
FR-13	F3	Ops	Control Tower	P5, P7	Predicted-state snapshot test
FR-14	F3	Ops, Planner	Control Tower	P5	Playwright
FR-15	F3	Ops	City Twin	P3, P5	Reroute integration test
FR-16	F4	Security, Admin	Trust Center	P8	Replay/forgery tests
FR-17	F4	Security	Trust Center	P7	Per-layer labelled attack tests
FR-18	F4	Security	Trust Center	P7	Reputation convergence test

FR	Feature	Persona	Prototype screen	Build phase	Verification
FR-19	F4	Ops, Security	Trust Center, Network Graph	P7	30% blackout chaos run
FR-20	F4	Security	Trust Center	P7	Chaos injection → detection E2E
FR-21	F5	Security	Fidelity Lab	P9	Prediction/actual join coverage
FR-22	F5	Security	Fidelity Lab	P9	Metric unit tests vs hand-computed values
FR-23	F5	Admin	Fidelity Lab	P9	Injected drift test
FR-24	F5	Security	Fidelity Lab	P9	Seeded benchmark reproduction
FR-25	F5	Planner, judges	Fidelity Lab	P9	Report diff on re-run

Appendix A — Scenario DSL

```
{
  "type": "port_closure | cyclone | road_flood | demand_spike | supplier_failure | strike | data_blackout",
  "target": "PORT_CHENNAI",
  "polygon": null,
  "start": "+6h",
  "duration_h": 120,
  "severity": 1.0,
  "params": {}
}
```

Appendix B — Trust layer reason codes (initial set)

SCHEMA_* (L1), HMAC_INVALID (L2), REPLAY_NONCE / STALE_TS / DUPLICATE_ID (L3), PHYSICS_TELEPORT / PHYSICS_ACCEL (L4), OFFROAD (L5), KALMAN_GATE (L6), TWIN_ENVELOPE (L7), ASN_OUTLIER / RECON_MISMATCH (L8), LOW_REPUTATION / SLA_SILENT (L9).